

mlp_week09

March 18, 2026

1 Week 9 - Introduction to Neural Networks

Yuting Xie (preferred: Ella Xie)

1.0.1 Aims

The main concepts covered in this notebook are:

- getting familiar with the basics of keras
- input-output with keras
- construction of neural network models with keras

This week, we will be exploring the basics of keras and building and fitting our first neural network with keras. For a quick introduction, please see https://keras.io/getting_started/intro_to_keras_for_engineers/

When completing worksheets:

- You will have tasks tagged by (CORE) and (EXTRA).
- Your primary aim is to complete the (CORE) components during the WS session, afterwards you can try to complete the (EXTRA) tasks for your self-learning process.

Instructions for submitting your workshops can be found at the end of worksheet. As a reminder, you must submit a pdf of your notebook on Learn by 16:00 PM on the Friday of the week the workshop was given.

2 Setup

2.1 Packages

Let's load the some of the packages needed for this workshop.

```
[1]: # Import necessary
!pip install tensorflow
import numpy as np
import tensorflow as tf
from tensorflow import keras
from tensorflow.keras import layers
import matplotlib.pyplot as plt
import seaborn as sns
```

```

Collecting tensorflow
  Using cached tensorflow-2.21.0-cp312-cp312-manylinux_2_27_x86_64.whl.metadata
(4.4 kB)
Collecting absl-py>=1.0.0 (from tensorflow)
  Using cached absl_py-2.4.0-py3-none-any.whl.metadata (3.3 kB)
Collecting astunparse>=1.6.0 (from tensorflow)
  Using cached astunparse-1.6.3-py2.py3-none-any.whl.metadata (4.4 kB)
Collecting flatbuffers>=25.9.23 (from tensorflow)
  Using cached flatbuffers-25.12.19-py2.py3-none-any.whl.metadata (1.0 kB)
Collecting gast!=0.5.0,!0.5.1,!0.5.2,>=0.2.1 (from tensorflow)
  Using cached gast-0.7.0-py3-none-any.whl.metadata (1.5 kB)
Collecting google_pasta>=0.1.1 (from tensorflow)
  Using cached google_pasta-0.2.0-py3-none-any.whl.metadata (814 bytes)
Collecting libclang>=13.0.0 (from tensorflow)
  Using cached libclang-18.1.1-py2.py3-none-manylinux2010_x86_64.whl.metadata
(5.2 kB)
Collecting opt_einsum>=2.3.2 (from tensorflow)
  Using cached opt_einsum-3.4.0-py3-none-any.whl.metadata (6.3 kB)
Requirement already satisfied: packaging in /opt/conda/lib/python3.12/site-
packages (from tensorflow) (25.0)
Collecting protobuf<8.0.0,>=6.31.1 (from tensorflow)
  Using cached protobuf-7.34.0-cp310-abi3-manylinux2014_x86_64.whl.metadata (595
bytes)
Requirement already satisfied: requests<3,>=2.21.0 in
/opt/conda/lib/python3.12/site-packages (from tensorflow) (2.32.5)
Requirement already satisfied: setuptools in /opt/conda/lib/python3.12/site-
packages (from tensorflow) (80.9.0)
Requirement already satisfied: six>=1.12.0 in /opt/conda/lib/python3.12/site-
packages (from tensorflow) (1.17.0)
Collecting termcolor>=1.1.0 (from tensorflow)
  Using cached termcolor-3.3.0-py3-none-any.whl.metadata (6.5 kB)
Requirement already satisfied: typing_extensions>=3.6.6 in
/opt/conda/lib/python3.12/site-packages (from tensorflow) (4.14.0)
Collecting wrapt>=1.11.0 (from tensorflow)
  Using cached wrapt-2.1.2-cp312-cp312-
manylinux1_x86_64.manylinux_2_28_x86_64.manylinux_2_5_x86_64.whl.metadata (7.4
kB)
Requirement already satisfied: grpcio<2.0,>=1.24.3 in
/opt/conda/lib/python3.12/site-packages (from tensorflow) (1.71.0)
Collecting keras>=3.12.0 (from tensorflow)
  Using cached keras-3.13.2-py3-none-any.whl.metadata (6.3 kB)
Requirement already satisfied: numpy>=1.26.0 in /opt/conda/lib/python3.12/site-
packages (from tensorflow) (1.26.4)
Collecting h5py<3.15.0,>=3.11.0 (from tensorflow)
  Using cached
h5py-3.14.0-cp312-cp312-manylinux_2_17_x86_64.manylinux2014_x86_64.whl.metadata
(2.7 kB)
Collecting ml_dtypes<1.0.0,>=0.5.1 (from tensorflow)

```

Using cached ml_dtypes-0.5.4-cp312-cp312-manylinux_2_27_x86_64.manylinux_2_28_x86_64.whl.metadata (8.9 kB)
Requirement already satisfied: charset_normalizer<4,>=2 in /opt/conda/lib/python3.12/site-packages (from requests<3,>=2.21.0->tensorflow) (3.4.2)
Requirement already satisfied: idna<4,>=2.5 in /opt/conda/lib/python3.12/site-packages (from requests<3,>=2.21.0->tensorflow) (3.10)
Requirement already satisfied: urllib3<3,>=1.21.1 in /opt/conda/lib/python3.12/site-packages (from requests<3,>=2.21.0->tensorflow) (2.6.3)
Requirement already satisfied: certifi>=2017.4.17 in /opt/conda/lib/python3.12/site-packages (from requests<3,>=2.21.0->tensorflow) (2025.6.15)
Requirement already satisfied: wheel<1.0,>=0.23.0 in /opt/conda/lib/python3.12/site-packages (from astunparse>=1.6.0->tensorflow) (0.45.1)
Requirement already satisfied: rich in /opt/conda/lib/python3.12/site-packages (from keras>=3.12.0->tensorflow) (14.3.2)
Collecting namex (from keras>=3.12.0->tensorflow)
Using cached namex-0.1.0-py3-none-any.whl.metadata (322 bytes)
Requirement already satisfied: optree in /opt/conda/lib/python3.12/site-packages (from keras>=3.12.0->tensorflow) (0.18.0)
Requirement already satisfied: markdown-it-py>=2.2.0 in /opt/conda/lib/python3.12/site-packages (from rich->keras>=3.12.0->tensorflow) (4.0.0)
Requirement already satisfied: pygments<3.0.0,>=2.13.0 in /opt/conda/lib/python3.12/site-packages (from rich->keras>=3.12.0->tensorflow) (2.19.1)
Requirement already satisfied: mdurl~=0.1 in /opt/conda/lib/python3.12/site-packages (from markdown-it-py>=2.2.0->rich->keras>=3.12.0->tensorflow) (0.1.2)
Using cached tensorflow-2.21.0-cp312-cp312-manylinux_2_27_x86_64.whl (572.6 MB)
Using cached
h5py-3.14.0-cp312-cp312-manylinux_2_17_x86_64.manylinux2014_x86_64.whl (4.9 MB)
Using cached
ml_dtypes-0.5.4-cp312-cp312-manylinux_2_27_x86_64.manylinux_2_28_x86_64.whl (5.0 MB)
Using cached protobuf-7.34.0-cp310-abi3-manylinux2014_x86_64.whl (324 kB)
Using cached absl_py-2.4.0-py3-none-any.whl (135 kB)
Using cached astunparse-1.6.3-py2.py3-none-any.whl (12 kB)
Using cached flatbuffers-25.12.19-py2.py3-none-any.whl (26 kB)
Using cached gast-0.7.0-py3-none-any.whl (22 kB)
Using cached google_pasta-0.2.0-py3-none-any.whl (57 kB)
Using cached keras-3.13.2-py3-none-any.whl (1.5 MB)
Using cached libclang-18.1.1-py2.py3-none-manylinux2010_x86_64.whl (24.5 MB)
Using cached opt_einsum-3.4.0-py3-none-any.whl (71 kB)
Using cached termcolor-3.3.0-py3-none-any.whl (7.7 kB)
Using cached wrapt-2.1.2-cp312-cp312-manylinux1_x86_64.manylinux_2_28_x86_64.manylinux_2_5_x86_64.whl (121 kB)

```

Using cached namex-0.1.0-py3-none-any.whl (5.9 kB)
Installing collected packages: namex, libclang, flatbuffers, wrapt, termcolor,
protobuf, opt_einsum, ml_dtypes, h5py, google_pasta, gast, astunparse, absl-py,
keras, tensorflow
  Attempting uninstall: protobuf
1/15 [libclang]
    Found existing installation: protobuf 5.29.3
1/15 [libclang]
    Uninstalling protobuf-5.29.3:
1/15 [libclang]
    Successfully uninstalled protobuf-5.29.3
5/15 [protobuf]
    Attempting uninstall: h5py0m
7/15 [ml_dtypes]
    Found existing installation: h5py 3.15.1
7/15 [ml_dtypes]
    Uninstalling h5py-3.15.1:1m
7/15 [ml_dtypes]
    Successfully uninstalled h5py-3.15.1m
8/15 [h5py]]
                                     15/15
[tensorflow]5 [tensorflow]a]

```

ERROR: pip's dependency resolver does not currently take into account all the packages that are installed. This behaviour is the source of the following dependency conflicts.

googleapis-common-protos 1.72.0 requires

protobuf!=4.21.1,!=4.21.2,!=4.21.3,!=4.21.4,!=4.21.5,<7.0.0,>=3.20.2, but you have protobuf 7.34.0 which is incompatible.

google-ai-generativelanguage 0.6.15 requires protobuf!=4.21.0,!=4.21.1,!=4.21.2,!=4.21.3,!=4.21.4,!=4.21.5,<6.0.0dev,>=3.20.2, but you have protobuf 7.34.0 which is incompatible.

google-api-core 2.29.0 requires protobuf!=3.20.0,!=3.20.1,!=4.21.0,!=4.21.1,!=4.21.2,!=4.21.3,!=4.21.4,!=4.21.5,<7.0.0,>=3.19.5, but you have protobuf 7.34.0 which is incompatible.

proto-plus 1.27.1 requires protobuf<7.0.0,>=3.19.0, but you have protobuf 7.34.0 which is incompatible.

Successfully installed absl-py-2.4.0 astunparse-1.6.3 flatbuffers-25.12.19 gast-0.7.0 google_pasta-0.2.0 h5py-3.14.0 keras-3.13.2 libclang-18.1.1 ml_dtypes-0.5.4 namex-0.1.0 opt_einsum-3.4.0 protobuf-7.34.0 tensorflow-2.21.0 termcolor-3.3.0 wrapt-2.1.2

WARNING: All log messages before absl::InitializeLog() is called are written to STDERR

I0000 00:00:1773856007.388441 707 port.cc:153] oneDNN custom operations are on. You may see slightly different numerical results due to floating-point round-off errors from different computation orders. To turn them off, set the environment variable `TF\_ENABLE\_ONEDNN\_OPTS=0`.

I0000 00:00:1773856007.389248 707 cudart_stub.cc:31] Could not find cuda drivers on your machine, GPU will not be used.

I0000 00:00:1773856007.448124 707 cpu_feature_guard.cc:227] This TensorFlow binary is optimized to use available CPU instructions in performance-critical operations.

To enable the following instructions: AVX2 AVX512F AVX512_VNNI FMA, in other operations, rebuild TensorFlow with the appropriate compiler flags.

WARNING: All log messages before absl::InitializeLog() is called are written to STDERR

I0000 00:00:1773856014.755466 707 port.cc:153] oneDNN custom operations are on. You may see slightly different numerical results due to floating-point round-off errors from different computation orders. To turn them off, set the environment variable `TF\_ENABLE\_ONEDNN\_OPTS=0`.

I0000 00:00:1773856014.755985 707 cudart_stub.cc:31] Could not find cuda drivers on your machine, GPU will not be used.

3 Part 1: Simulated Data

In the first part of this workshop, we will work with a simple simulated example to gain some basic understanding of neural networks and keras.

We generate $N = 2000$ simulated data points. First we generate the labels assuming that:

$$y_i \sim \text{Bern}(0.5).$$

Next, we generate the features of dimension $D = 2$. The first feature is simulated uniformly in $[-\pi, \pi]$

$$x_{i,1} \sim \text{Uniform}(-\pi, \pi).$$

For the second feature, if $y = 0$, then

$$x_{i,2} \sim \text{Norm}(\cos(x_{i,1}), \sigma^2).$$

Otherwise, if $y = 1$, then

$$x_{i,2} \sim \text{Norm}(1 + \cos(x_{i,1}), \sigma^2).$$

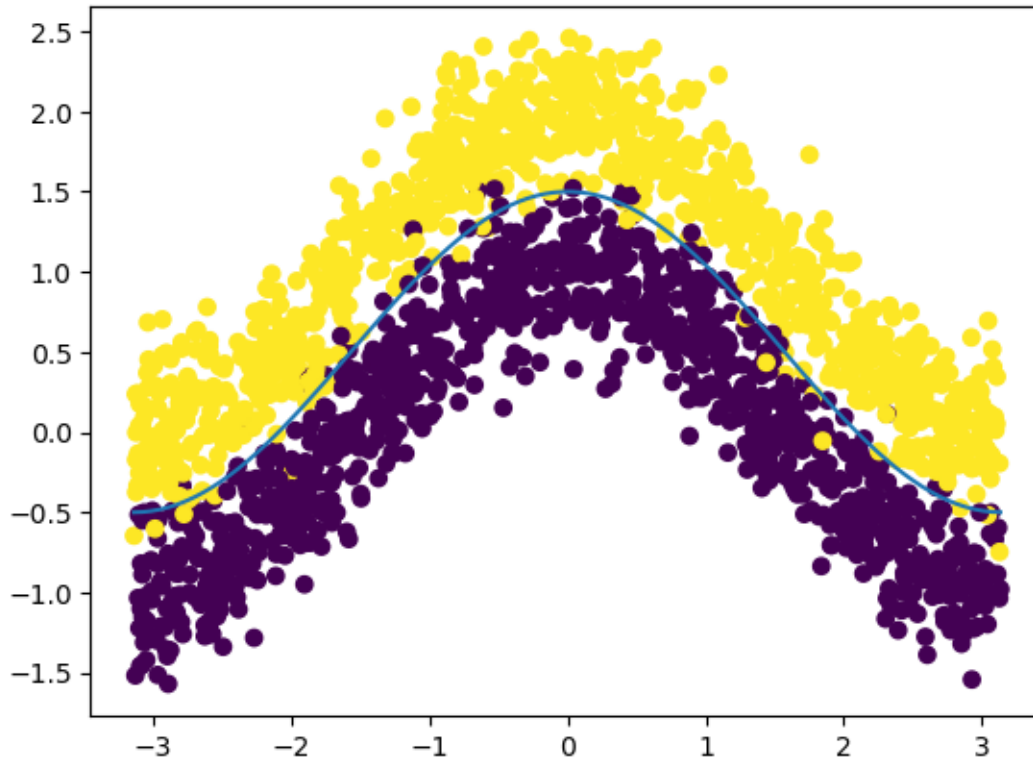
```
[2]: # Generate data
keras.utils.set_random_seed(11205)
N = 2000
D = 2
y = np.random.binomial(1, 0.5, N)
X = np.zeros((N, D))
X[:,0] = np.random.uniform(-np.pi, np.pi, N)
X[y == 0,1] = np.cos(X[y == 0,0]) + np.random.normal(0, 0.25, (y == 0).sum())
X[y == 1,1] = 1 + np.cos(X[y == 1,0]) + np.random.normal(0, 0.25, (y == 1).
↪sum())
```

3.0.1 Exercise 1 (CORE)

Plot the data colored by the class labels.

EXTRA. Derive the true decision boundary and add it to the plot.

```
[3]: # Code for your answer here!
plt.scatter(X[:,0],X[:,1],c=y)
x_boun=np.linspace(X[:,0].min(),X[:,0].max(),100)
y_boun=0.5+np.cos(x_boun)
plt.plot(x_boun,y_boun)
plt.show()
```



EXTRA. as shown above

3.1 Building a baseline logistic regression in keras

Let's start by building a logistic regression model in keras, using the `Functional` model: https://keras.io/guides/functional_api/

A `Functional` model provides a way to build a directed graph of `Layers` that connect to each other. `Layers` are the basic building blocks of neural networks in keras: <https://keras.io/api/layers/>

To construct a logistic regression model, we only need two layers: - an input layer, which we construct using an `Input` layer: https://keras.io/api/layers/core_layers/input/ - an output layer, which we construct using a `Dense` layer: https://keras.io/api/layers/core_layers/dense/

```
[4]: # Defining the layers of the model
input_layer = keras.layers.Input(shape=(D,))
logit_layer = keras.layers.Dense(1, activation='sigmoid')
```

Then, we combine the layers and create our model which connects the input and output.

```
[5]: output = logit_layer(input_layer)
model_lr = keras.models.Model(input_layer, output)
model_lr.summary()
```

```
E0000 00:00:1773856017.309235      707 cuda_platform.cc:52] failed call to
cuInit: INTERNAL: CUDA error: Failed call to cuInit: UNKNOWN ERROR (303)
```

Model: "functional"

Layer (type)	Output Shape	Param #
input_layer (InputLayer)	(None , 2)	0
dense (Dense)	(None , 1)	3

Total params: 3 (12.00 B)

Trainable params: 3 (12.00 B)

Non-trainable params: 0 (0.00 B)

```
[6]: # Print the number of layers in the model
print(len(model_lr.layers))
```

2

Alternatively, we can build the model using the **Sequential** model, see <https://keras.io/api/models/sequential/> and https://keras.io/guides/sequential_model/. A **Sequential** model is essentially a plain stack of **Layers** where each layer has exactly one input tensor and one output tensor, while a **Functional** model allows more general constructions (i.e. a graph of layers).

```
[7]: model_lr2 = keras.Sequential(
    [
        keras.Input(shape=(D,)),
        keras.layers.Dense(1, activation='sigmoid'),
    ]
)
model_lr2.summary()
```

Model: "sequential"

Layer (type)	Output Shape	Param #
dense_1 (Dense)	(None , 1)	3

Total params: 3 (12.00 B)

Trainable params: 3 (12.00 B)

Non-trainable params: 0 (0.00 B)

Now, we are ready to compile the model: https://keras.io/api/models/model_training_apis/

When compiling, we need to specify: - a loss function: <https://keras.io/api/losses/> - an optimizer: <https://keras.io/api/optimizers/> - (optionally) metrics: <https://keras.io/api/metrics/>

```
[8]: model_lr.compile(
    loss='binary_crossentropy', # or loss=keras.losses.
    ↪BinaryCrossentropy(from_logits=False),
    optimizer='adam', # or optimizer=keras.optimizers.Adam(learning_rate=1e-3),
    metrics=[
        keras.metrics.BinaryAccuracy(name='accuracy'),
        keras.metrics.AUC(name='auc')
    ],
)
```

3.1.1 Training the model

Now, we are ready to fit the model! We need to specify: - the number of **epochs** (which is the number of times that we cycle through the whole training data during gradient descent),

and optionally various other parameters, such as

- **shuffle** (e.g. `shuffle=True`), which specifies whether to shuffle the training data before each epoch.
- **validation_split** which is a float between 0 and 1, determining the fraction of the training data to be used as validation data. The model will set apart this fraction of the training data, will not train on it, and will evaluate the loss and any model metrics on this data at the end of each epoch. This is useful for **early stopping**, i.e. determining if training should be stopped early to prevent overfitting.
- **batch_size**: Integer or None. Number of samples per gradient update. If unspecified, `batch_size` will default to 32. For more details, see: https://keras.io/api/models/model_training_apis/

Caution: the data is not shuffled before splitting the data into a validation set. If the data are ordered in particular way when stored, then the training and validation sets can potentially characterize different populations. Thus, it is recommend to first shuffle the order of your data before fitting.

```
[9]: # If you need to shuffle the data before fitting uncomment the following lines
# from sklearn.utils import shuffle
# X, y = shuffle(X, y, random_state=11205)
```

```
[10]: model_lr.fit(x=X, y=y, epochs=50, shuffle=True, validation_split=0.1)
```

Epoch 1/50

57/57 1s 9ms/step -
accuracy: 0.5272 - auc: 0.5536 - loss: 0.8589 - val_accuracy: 0.5250 - val_auc:
0.5678 - val_loss: 0.8517

Epoch 2/50

57/57 0s 3ms/step -
accuracy: 0.5344 - auc: 0.5721 - loss: 0.8220 - val_accuracy: 0.5350 - val_auc:
0.5887 - val_loss: 0.8177

Epoch 3/50

57/57 0s 4ms/step -
accuracy: 0.5389 - auc: 0.5955 - loss: 0.7877 - val_accuracy: 0.5300 - val_auc:
0.6126 - val_loss: 0.7864

Epoch 4/50

57/57 0s 3ms/step -
accuracy: 0.5411 - auc: 0.6241 - loss: 0.7562 - val_accuracy: 0.5300 - val_auc:
0.6355 - val_loss: 0.7576

Epoch 5/50

57/57 0s 3ms/step -
accuracy: 0.5472 - auc: 0.6561 - loss: 0.7275 - val_accuracy: 0.5250 - val_auc:
0.6572 - val_loss: 0.7315

Epoch 6/50

57/57 0s 3ms/step -
accuracy: 0.5483 - auc: 0.6863 - loss: 0.7014 - val_accuracy: 0.5150 - val_auc:
0.6845 - val_loss: 0.7080

Epoch 7/50

57/57 0s 3ms/step -
accuracy: 0.5522 - auc: 0.7112 - loss: 0.6781 - val_accuracy: 0.5100 - val_auc:
0.7005 - val_loss: 0.6872

Epoch 8/50

57/57 0s 3ms/step -
accuracy: 0.5511 - auc: 0.7312 - loss: 0.6574 - val_accuracy: 0.5050 - val_auc:
0.7139 - val_loss: 0.6688

Epoch 9/50

57/57 0s 3ms/step -
accuracy: 0.5572 - auc: 0.7467 - loss: 0.6392 - val_accuracy: 0.5200 - val_auc:
0.7224 - val_loss: 0.6528

Epoch 10/50

57/57 0s 3ms/step -
accuracy: 0.5761 - auc: 0.7580 - loss: 0.6233 - val_accuracy: 0.5500 - val_auc:
0.7301 - val_loss: 0.6390

Epoch 11/50

57/57 0s 4ms/step -
accuracy: 0.6078 - auc: 0.7652 - loss: 0.6095 - val_accuracy: 0.5850 - val_auc:
0.7387 - val_loss: 0.6271

Epoch 12/50

57/57 0s 3ms/step -
accuracy: 0.6361 - auc: 0.7698 - loss: 0.5977 - val_accuracy: 0.6150 - val_auc:
0.7422 - val_loss: 0.6171
Epoch 13/50
57/57 0s 4ms/step -
accuracy: 0.6550 - auc: 0.7731 - loss: 0.5875 - val_accuracy: 0.6300 - val_auc:
0.7437 - val_loss: 0.6086
Epoch 14/50
57/57 0s 3ms/step -
accuracy: 0.6694 - auc: 0.7768 - loss: 0.5789 - val_accuracy: 0.6450 - val_auc:
0.7460 - val_loss: 0.6015
Epoch 15/50
57/57 0s 3ms/step -
accuracy: 0.6850 - auc: 0.7809 - loss: 0.5716 - val_accuracy: 0.6500 - val_auc:
0.7476 - val_loss: 0.5955
Epoch 16/50
57/57 0s 3ms/step -
accuracy: 0.6867 - auc: 0.7850 - loss: 0.5653 - val_accuracy: 0.6550 - val_auc:
0.7527 - val_loss: 0.5905
Epoch 17/50
57/57 0s 3ms/step -
accuracy: 0.6867 - auc: 0.7885 - loss: 0.5601 - val_accuracy: 0.6600 - val_auc:
0.7557 - val_loss: 0.5864
Epoch 18/50
57/57 0s 3ms/step -
accuracy: 0.6811 - auc: 0.7916 - loss: 0.5556 - val_accuracy: 0.6700 - val_auc:
0.7575 - val_loss: 0.5830
Epoch 19/50
57/57 0s 3ms/step -
accuracy: 0.6806 - auc: 0.7945 - loss: 0.5519 - val_accuracy: 0.6700 - val_auc:
0.7571 - val_loss: 0.5802
Epoch 20/50
57/57 0s 3ms/step -
accuracy: 0.6778 - auc: 0.7964 - loss: 0.5487 - val_accuracy: 0.6600 - val_auc:
0.7578 - val_loss: 0.5778
Epoch 21/50
57/57 0s 3ms/step -
accuracy: 0.6783 - auc: 0.7977 - loss: 0.5459 - val_accuracy: 0.6550 - val_auc:
0.7587 - val_loss: 0.5759
Epoch 22/50
57/57 0s 3ms/step -
accuracy: 0.6772 - auc: 0.7986 - loss: 0.5436 - val_accuracy: 0.6550 - val_auc:
0.7608 - val_loss: 0.5742
Epoch 23/50
57/57 0s 3ms/step -
accuracy: 0.6767 - auc: 0.7994 - loss: 0.5416 - val_accuracy: 0.6550 - val_auc:
0.7622 - val_loss: 0.5729
Epoch 24/50

57/57 0s 3ms/step -
 accuracy: 0.6767 - auc: 0.7999 - loss: 0.5398 - val_accuracy: 0.6550 - val_auc:
 0.7629 - val_loss: 0.5717
 Epoch 25/50
 57/57 0s 3ms/step -
 accuracy: 0.6739 - auc: 0.8003 - loss: 0.5383 - val_accuracy: 0.6500 - val_auc:
 0.7633 - val_loss: 0.5708
 Epoch 26/50
 57/57 0s 3ms/step -
 accuracy: 0.6722 - auc: 0.8006 - loss: 0.5370 - val_accuracy: 0.6500 - val_auc:
 0.7634 - val_loss: 0.5699
 Epoch 27/50
 57/57 0s 3ms/step -
 accuracy: 0.6722 - auc: 0.8008 - loss: 0.5359 - val_accuracy: 0.6400 - val_auc:
 0.7632 - val_loss: 0.5692
 Epoch 28/50
 57/57 0s 4ms/step -
 accuracy: 0.6750 - auc: 0.8009 - loss: 0.5348 - val_accuracy: 0.6400 - val_auc:
 0.7629 - val_loss: 0.5686
 Epoch 29/50
 57/57 0s 4ms/step -
 accuracy: 0.6750 - auc: 0.8010 - loss: 0.5339 - val_accuracy: 0.6450 - val_auc:
 0.7634 - val_loss: 0.5681
 Epoch 30/50
 57/57 0s 4ms/step -
 accuracy: 0.6756 - auc: 0.8011 - loss: 0.5331 - val_accuracy: 0.6450 - val_auc:
 0.7630 - val_loss: 0.5677
 Epoch 31/50
 57/57 0s 4ms/step -
 accuracy: 0.6767 - auc: 0.8012 - loss: 0.5324 - val_accuracy: 0.6450 - val_auc:
 0.7632 - val_loss: 0.5673
 Epoch 32/50
 57/57 0s 3ms/step -
 accuracy: 0.6778 - auc: 0.8011 - loss: 0.5318 - val_accuracy: 0.6450 - val_auc:
 0.7630 - val_loss: 0.5670
 Epoch 33/50
 57/57 0s 3ms/step -
 accuracy: 0.6778 - auc: 0.8012 - loss: 0.5312 - val_accuracy: 0.6500 - val_auc:
 0.7633 - val_loss: 0.5667
 Epoch 34/50
 57/57 0s 3ms/step -
 accuracy: 0.6778 - auc: 0.8012 - loss: 0.5306 - val_accuracy: 0.6500 - val_auc:
 0.7637 - val_loss: 0.5665
 Epoch 35/50
 57/57 0s 3ms/step -
 accuracy: 0.6767 - auc: 0.8012 - loss: 0.5302 - val_accuracy: 0.6400 - val_auc:
 0.7635 - val_loss: 0.5663
 Epoch 36/50

57/57 0s 3ms/step -
accuracy: 0.6767 - auc: 0.8014 - loss: 0.5297 - val_accuracy: 0.6400 - val_auc:
0.7630 - val_loss: 0.5661
Epoch 37/50

57/57 0s 3ms/step -
accuracy: 0.6767 - auc: 0.8014 - loss: 0.5294 - val_accuracy: 0.6400 - val_auc:
0.7627 - val_loss: 0.5659
Epoch 38/50

57/57 0s 3ms/step -
accuracy: 0.6772 - auc: 0.8012 - loss: 0.5290 - val_accuracy: 0.6400 - val_auc:
0.7634 - val_loss: 0.5658
Epoch 39/50

57/57 0s 3ms/step -
accuracy: 0.6767 - auc: 0.8013 - loss: 0.5287 - val_accuracy: 0.6400 - val_auc:
0.7635 - val_loss: 0.5657
Epoch 40/50

57/57 0s 3ms/step -
accuracy: 0.6750 - auc: 0.8013 - loss: 0.5284 - val_accuracy: 0.6300 - val_auc:
0.7635 - val_loss: 0.5656
Epoch 41/50

57/57 0s 3ms/step -
accuracy: 0.6756 - auc: 0.8013 - loss: 0.5282 - val_accuracy: 0.6250 - val_auc:
0.7632 - val_loss: 0.5655
Epoch 42/50

57/57 0s 3ms/step -
accuracy: 0.6744 - auc: 0.8013 - loss: 0.5279 - val_accuracy: 0.6250 - val_auc:
0.7629 - val_loss: 0.5654
Epoch 43/50

57/57 0s 3ms/step -
accuracy: 0.6744 - auc: 0.8012 - loss: 0.5277 - val_accuracy: 0.6250 - val_auc:
0.7629 - val_loss: 0.5654
Epoch 44/50

57/57 0s 3ms/step -
accuracy: 0.6744 - auc: 0.8013 - loss: 0.5275 - val_accuracy: 0.6250 - val_auc:
0.7634 - val_loss: 0.5653
Epoch 45/50

57/57 0s 3ms/step -
accuracy: 0.6750 - auc: 0.8013 - loss: 0.5274 - val_accuracy: 0.6250 - val_auc:
0.7630 - val_loss: 0.5653
Epoch 46/50

57/57 0s 3ms/step -
accuracy: 0.6750 - auc: 0.8013 - loss: 0.5272 - val_accuracy: 0.6250 - val_auc:
0.7630 - val_loss: 0.5653
Epoch 47/50

57/57 0s 4ms/step -
accuracy: 0.6750 - auc: 0.8013 - loss: 0.5271 - val_accuracy: 0.6250 - val_auc:
0.7627 - val_loss: 0.5653
Epoch 48/50

```

57/57          0s 3ms/step -
accuracy: 0.6761 - auc: 0.8012 - loss: 0.5269 - val_accuracy: 0.6250 - val_auc:
0.7628 - val_loss: 0.5653
Epoch 49/50
57/57          0s 3ms/step -
accuracy: 0.6772 - auc: 0.8013 - loss: 0.5268 - val_accuracy: 0.6200 - val_auc:
0.7629 - val_loss: 0.5653
Epoch 50/50
57/57          0s 3ms/step -
accuracy: 0.6767 - auc: 0.8012 - loss: 0.5267 - val_accuracy: 0.6200 - val_auc:
0.7630 - val_loss: 0.5653

```

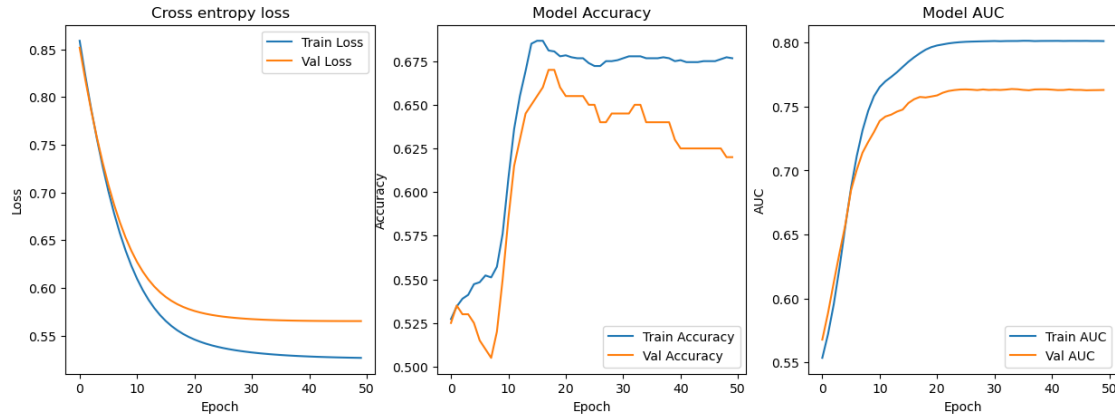
[10]: <keras.src.callbacks.history.History at 0x7fc9660dd1c0>

Let's plot the metrics as a function of the number of epochs.

```

[11]: # Plot the training history
history = model_lr.history.history
fig, ax = plt.subplots(1, 3, figsize=(15, 5))
ax[0].plot(history['loss'], label='Train Loss')
ax[0].plot(history['val_loss'], label='Val Loss')
ax[0].set_title('Cross entropy loss')
ax[0].set_xlabel('Epoch')
ax[0].set_ylabel('Loss')
ax[0].legend()
ax[1].plot(history['accuracy'], label='Train Accuracy')
ax[1].plot(history['val_accuracy'], label='Val Accuracy')
ax[1].set_title('Model Accuracy')
ax[1].set_xlabel('Epoch')
ax[1].set_ylabel('Accuracy')
ax[1].legend()
ax[2].plot(history['auc'], label='Train AUC')
ax[2].plot(history['val_auc'], label='Val AUC')
ax[2].set_title('Model AUC')
ax[2].set_xlabel('Epoch')
ax[2].set_ylabel('AUC')
ax[2].legend()
plt.show()

```



If you notice that the validation loss (and metrics) is still decreasing (increasing) as a function of epochs, this suggests that you may need to train the model longer.

If you notice that the validation loss (and metrics) starts to increase (decrease) as a function of epochs, this suggests that the model is starting to overfit. Based on this, we can determine the number of epochs to use based on the U-shape of the validation loss (this is known as early stopping, for more info see: https://keras.io/api/callbacks/early_stopping/).

3.1.2 Testing the model

Now, we can use `.predict()` to make predictions. Let's predict at a grid of test points and draw a heatmap of the predicted probabilities.

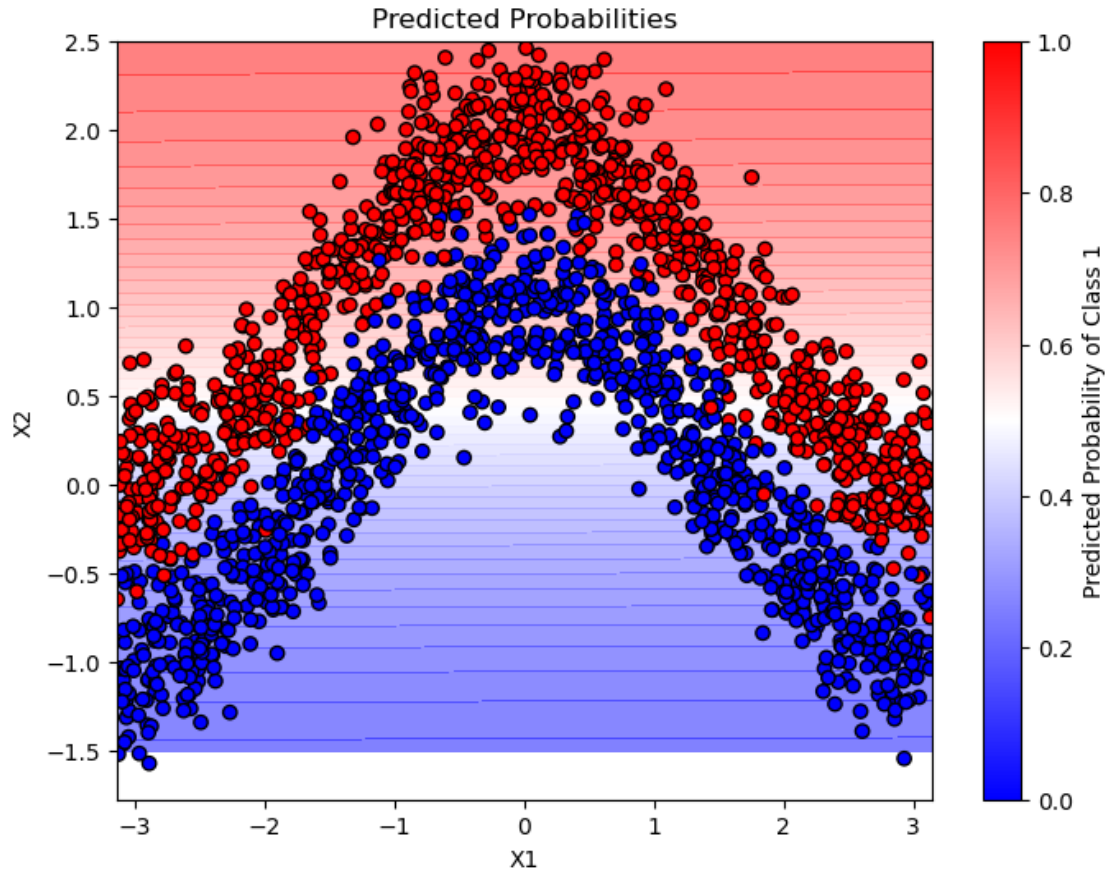
```
[12]: # Create a grid of test points
X_grid1, X_grid2 = np.meshgrid(np.linspace(-np.pi, np.pi, 100), np.linspace(-1.
    ↪5, 2.5, 100))
X_grid = np.column_stack([X_grid1.ravel(), X_grid2.ravel()])

# Predict probabilities
y_pred_prob_lr = model_lr.predict(X_grid)

# Draw a heatmap of the predicted probabilities
plt.figure(figsize=(8, 6))
plt.contourf(X_grid1, X_grid2, y_pred_prob_lr.reshape(X_grid1.shape),
    ↪levels=50, cmap='bwr', alpha=0.5)
plt.scatter(X[:,0], X[:,1], c=y, cmap='bwr', edgecolor='k')
plt.xlabel('X1')
plt.ylabel('X2')
plt.title('Predicted Probabilities')
plt.colorbar(label='Predicted Probability of Class 1')
plt.show()
```

313/313

0s 926us/step



3.1.3 Exercise 2 (CORE)

- Generate 1000 test data points from the same data generating process used to generate the training data.
- Compute the `classification_report` and `confusion_matrix`
- Is the result what you expect, and why?

```
[13]: # Code for your answer here!
from sklearn.metrics import classification_report, confusion_matrix
N_test = 1000
y_test = np.random.binomial(1, 0.5, N_test)
X_test = np.zeros((N_test, D))
X_test[:,0] = np.random.uniform(-np.pi, np.pi, N_test)
X_test[y_test == 0,1] = np.cos(X_test[y_test == 0,0]) + np.random.normal(0, 0.
    ↪25, (y_test == 0).sum())
X_test[y_test == 1,1] = 1 + np.cos(X_test[y_test == 1,0]) + np.random.normal(0, 0.
    ↪0.25, (y_test == 1).sum())
y_pred_prob_test=model_lr.predict(X_test)
y_pred_test=(y_pred_prob_test>0.5).astype(int)
```



```
print(classification_report(y_test,y_pred_test))
print(confusion_matrix(y_test,y_pred_test))
```

```
32/32          0s 1ms/step
          precision    recall  f1-score   support

         0          0.68        0.67        0.68         496
         1          0.68        0.70        0.69         504

 accuracy                   0.68         1000
 macro avg          0.68        0.68        0.68         1000
weighted avg          0.68        0.68        0.68         1000
```

```
[[332 164]
 [153 351]]
```

yes. because the accuracy>50% shows it learn some pattern.

3.1.4 Adding a hidden layer

Now, let's add a hidden layer, with 2 neurons.

```
[14]: input_layer = keras.Input(shape=(D,))
      hidden_layer = keras.layers.Dense(2, activation='relu')(input_layer)
      output_layer = keras.layers.Dense(1, activation='sigmoid')(hidden_layer)
      model_hidden2 = keras.Model(inputs=input_layer, outputs=output_layer)
      model_hidden2.summary()
```

Model: "functional_2"

Layer (type)	Output Shape	Param #
input_layer_2 (InputLayer)	(None, 2)	0
dense_2 (Dense)	(None, 2)	6
dense_3 (Dense)	(None, 1)	3

Total params: 9 (36.00 B)

Trainable params: 9 (36.00 B)

Non-trainable params: 0 (0.00 B)

3.1.5 Exercise 3 (CORE)

Compile the model, train and plot the metrics as a function of epochs.

Continue to train the model for more epochs as needed. Note: rerun the fit line will continue to train the model from where it left off.

```
[15]: # Code for your answer here!
model_hidden2.
    ↪ compile(optimizer='adam', loss='binary_crossentropy', metrics=['accuracy'])
sequen=np.random.permutation(len(X))
X_sto=X[sequen]
y_sto=y[sequen]
train=model_hidden2.fit(X_sto,y_sto,epochs=50,batch_size=30,validation_split=0.
    ↪ 3,shuffle=True)
plt.plot(train.history['accuracy'], label='train accuracy')
plt.plot(train.history['val_accuracy'], label='val accuracy')
plt.legend()
plt.show()

plt.plot(train.history['loss'], label='train loss')
plt.plot(train.history['val_loss'], label='val loss')
plt.legend()
plt.show()
```

Epoch 1/50

47/47 1s 7ms/step -

accuracy: 0.5236 - loss: 0.7639 - val_accuracy: 0.5717 - val_loss: 0.7321

Epoch 2/50

23/47 0s 3ms/step -

accuracy: 0.5266 - loss: 0.7716

47/47 0s 6ms/step -

accuracy: 0.5293 - loss: 0.7420 - val_accuracy: 0.5683 - val_loss: 0.7164

Epoch 3/50

47/47 0s 5ms/step -

accuracy: 0.5243 - loss: 0.7254 - val_accuracy: 0.5750 - val_loss: 0.7044

Epoch 4/50

47/47 0s 4ms/step -

accuracy: 0.5271 - loss: 0.7120 - val_accuracy: 0.5733 - val_loss: 0.6946

Epoch 5/50

47/47 0s 4ms/step -

accuracy: 0.5329 - loss: 0.7005 - val_accuracy: 0.5650 - val_loss: 0.6862

Epoch 6/50

47/47 0s 4ms/step -

accuracy: 0.5321 - loss: 0.6899 - val_accuracy: 0.5683 - val_loss: 0.6783

Epoch 7/50

47/47 0s 4ms/step -

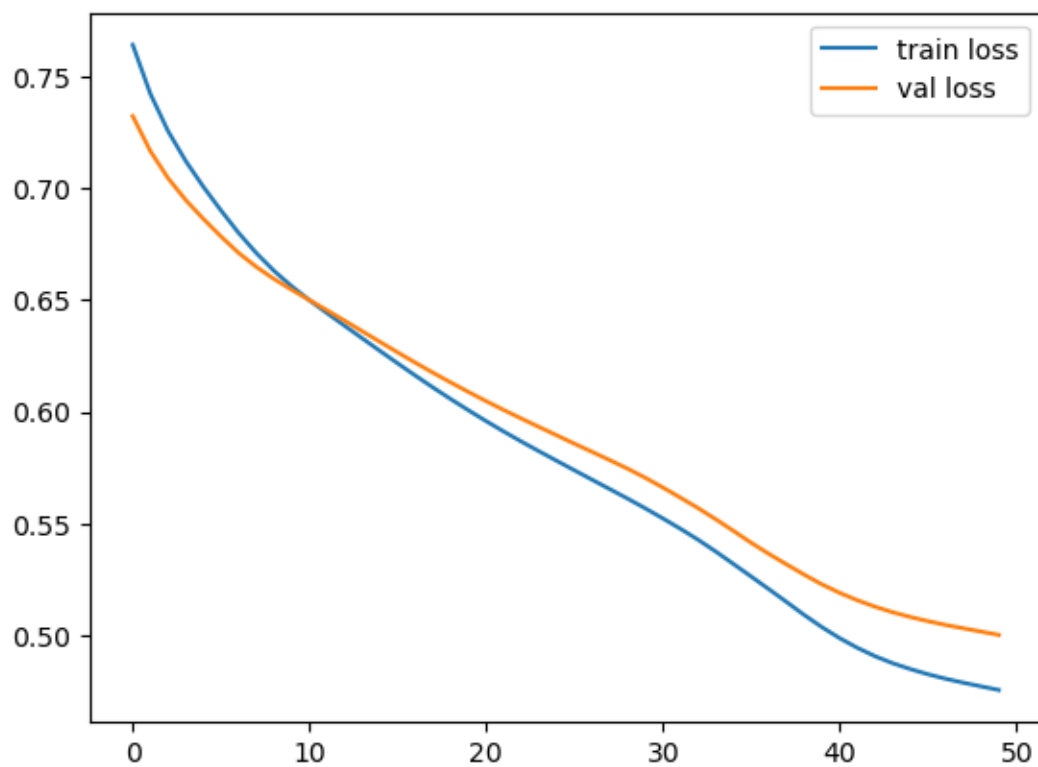
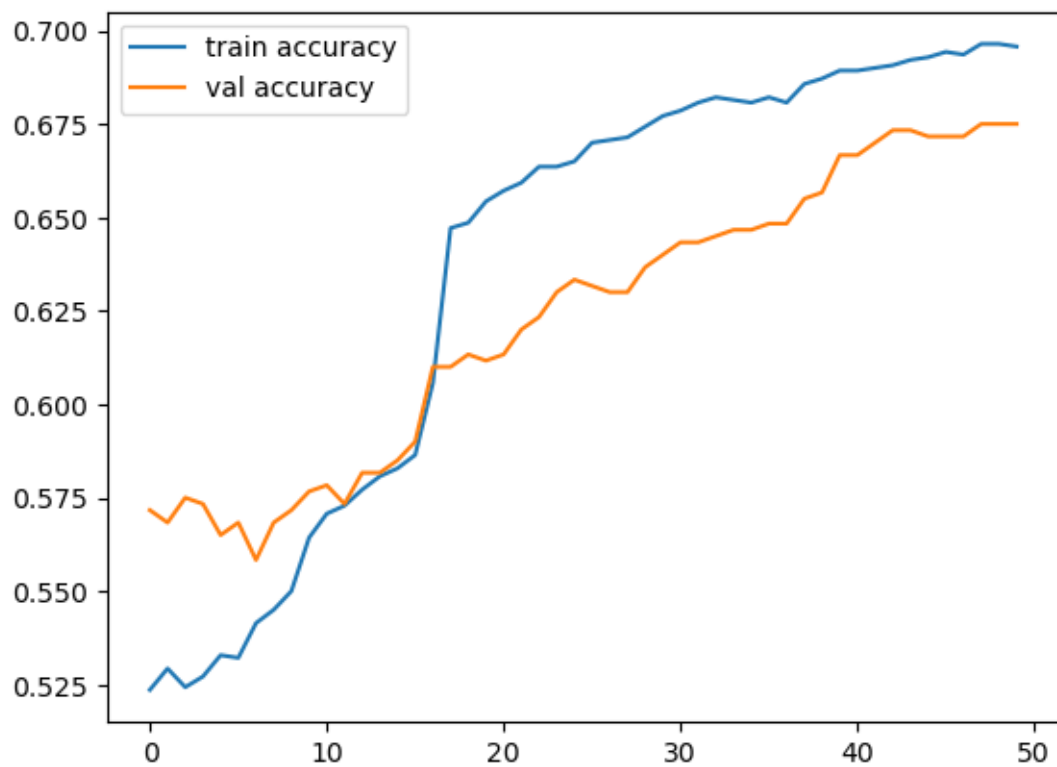
accuracy: 0.5414 - loss: 0.6799 - val_accuracy: 0.5583 - val_loss: 0.6709
 Epoch 8/50
 47/47 0s 4ms/step -
 accuracy: 0.5450 - loss: 0.6709 - val_accuracy: 0.5683 - val_loss: 0.6647
 Epoch 9/50
 47/47 0s 4ms/step -
 accuracy: 0.5500 - loss: 0.6629 - val_accuracy: 0.5717 - val_loss: 0.6593
 Epoch 10/50
 47/47 0s 4ms/step -
 accuracy: 0.5643 - loss: 0.6561 - val_accuracy: 0.5767 - val_loss: 0.6545
 Epoch 11/50
 47/47 0s 3ms/step -
 accuracy: 0.5707 - loss: 0.6499 - val_accuracy: 0.5783 - val_loss: 0.6499
 Epoch 12/50
 47/47 0s 3ms/step -
 accuracy: 0.5729 - loss: 0.6441 - val_accuracy: 0.5733 - val_loss: 0.6453
 Epoch 13/50
 47/47 0s 3ms/step -
 accuracy: 0.5771 - loss: 0.6384 - val_accuracy: 0.5817 - val_loss: 0.6407
 Epoch 14/50
 47/47 0s 4ms/step -
 accuracy: 0.5807 - loss: 0.6328 - val_accuracy: 0.5817 - val_loss: 0.6360
 Epoch 15/50
 47/47 0s 4ms/step -
 accuracy: 0.5829 - loss: 0.6272 - val_accuracy: 0.5850 - val_loss: 0.6312
 Epoch 16/50
 47/47 0s 4ms/step -
 accuracy: 0.5864 - loss: 0.6216 - val_accuracy: 0.5900 - val_loss: 0.6265
 Epoch 17/50
 47/47 0s 4ms/step -
 accuracy: 0.6057 - loss: 0.6162 - val_accuracy: 0.6100 - val_loss: 0.6219
 Epoch 18/50
 47/47 0s 4ms/step -
 accuracy: 0.6471 - loss: 0.6108 - val_accuracy: 0.6100 - val_loss: 0.6175
 Epoch 19/50
 47/47 0s 4ms/step -
 accuracy: 0.6486 - loss: 0.6057 - val_accuracy: 0.6133 - val_loss: 0.6131
 Epoch 20/50
 47/47 0s 4ms/step -
 accuracy: 0.6543 - loss: 0.6007 - val_accuracy: 0.6117 - val_loss: 0.6089
 Epoch 21/50
 47/47 0s 4ms/step -
 accuracy: 0.6571 - loss: 0.5959 - val_accuracy: 0.6133 - val_loss: 0.6048
 Epoch 22/50
 47/47 0s 3ms/step -
 accuracy: 0.6593 - loss: 0.5912 - val_accuracy: 0.6200 - val_loss: 0.6008
 Epoch 23/50
 47/47 0s 3ms/step -

accuracy: 0.6636 - loss: 0.5867 - val_accuracy: 0.6233 - val_loss: 0.5970
 Epoch 24/50
 47/47 0s 3ms/step -
 accuracy: 0.6636 - loss: 0.5824 - val_accuracy: 0.6300 - val_loss: 0.5933
 Epoch 25/50
 47/47 0s 4ms/step -
 accuracy: 0.6650 - loss: 0.5781 - val_accuracy: 0.6333 - val_loss: 0.5895
 Epoch 26/50
 47/47 0s 4ms/step -
 accuracy: 0.6700 - loss: 0.5739 - val_accuracy: 0.6317 - val_loss: 0.5858
 Epoch 27/50
 47/47 0s 3ms/step -
 accuracy: 0.6707 - loss: 0.5697 - val_accuracy: 0.6300 - val_loss: 0.5821
 Epoch 28/50
 47/47 0s 4ms/step -
 accuracy: 0.6714 - loss: 0.5655 - val_accuracy: 0.6300 - val_loss: 0.5784
 Epoch 29/50
 47/47 0s 4ms/step -
 accuracy: 0.6743 - loss: 0.5613 - val_accuracy: 0.6367 - val_loss: 0.5747
 Epoch 30/50
 47/47 0s 4ms/step -
 accuracy: 0.6771 - loss: 0.5569 - val_accuracy: 0.6400 - val_loss: 0.5706
 Epoch 31/50
 47/47 0s 5ms/step -
 accuracy: 0.6786 - loss: 0.5524 - val_accuracy: 0.6433 - val_loss: 0.5662
 Epoch 32/50
 47/47 0s 4ms/step -
 accuracy: 0.6807 - loss: 0.5478 - val_accuracy: 0.6433 - val_loss: 0.5617
 Epoch 33/50
 47/47 0s 7ms/step -
 accuracy: 0.6821 - loss: 0.5429 - val_accuracy: 0.6450 - val_loss: 0.5570
 Epoch 34/50
 47/47 0s 6ms/step -
 accuracy: 0.6814 - loss: 0.5376 - val_accuracy: 0.6467 - val_loss: 0.5520
 Epoch 35/50
 47/47 0s 7ms/step -
 accuracy: 0.6807 - loss: 0.5320 - val_accuracy: 0.6467 - val_loss: 0.5468
 Epoch 36/50
 47/47 0s 5ms/step -
 accuracy: 0.6821 - loss: 0.5264 - val_accuracy: 0.6483 - val_loss: 0.5415
 Epoch 37/50
 47/47 0s 4ms/step -
 accuracy: 0.6807 - loss: 0.5208 - val_accuracy: 0.6483 - val_loss: 0.5366
 Epoch 38/50
 47/47 0s 5ms/step -
 accuracy: 0.6857 - loss: 0.5151 - val_accuracy: 0.6550 - val_loss: 0.5319
 Epoch 39/50
 47/47 0s 6ms/step -

```

accuracy: 0.6871 - loss: 0.5093 - val_accuracy: 0.6567 - val_loss: 0.5273
Epoch 40/50
47/47          0s 5ms/step -
accuracy: 0.6893 - loss: 0.5039 - val_accuracy: 0.6667 - val_loss: 0.5230
Epoch 41/50
47/47          0s 4ms/step -
accuracy: 0.6893 - loss: 0.4990 - val_accuracy: 0.6667 - val_loss: 0.5192
Epoch 42/50
47/47          0s 4ms/step -
accuracy: 0.6900 - loss: 0.4947 - val_accuracy: 0.6700 - val_loss: 0.5159
Epoch 43/50
47/47          0s 5ms/step -
accuracy: 0.6907 - loss: 0.4909 - val_accuracy: 0.6733 - val_loss: 0.5130
Epoch 44/50
47/47          0s 4ms/step -
accuracy: 0.6921 - loss: 0.4878 - val_accuracy: 0.6733 - val_loss: 0.5106
Epoch 45/50
47/47          0s 4ms/step -
accuracy: 0.6929 - loss: 0.4852 - val_accuracy: 0.6717 - val_loss: 0.5084
Epoch 46/50
47/47          0s 3ms/step -
accuracy: 0.6943 - loss: 0.4829 - val_accuracy: 0.6717 - val_loss: 0.5066
Epoch 47/50
47/47          0s 3ms/step -
accuracy: 0.6936 - loss: 0.4809 - val_accuracy: 0.6717 - val_loss: 0.5049
Epoch 48/50
47/47          0s 3ms/step -
accuracy: 0.6964 - loss: 0.4791 - val_accuracy: 0.6750 - val_loss: 0.5034
Epoch 49/50
47/47          0s 3ms/step -
accuracy: 0.6964 - loss: 0.4775 - val_accuracy: 0.6750 - val_loss: 0.5019
Epoch 50/50
47/47          0s 3ms/step -
accuracy: 0.6957 - loss: 0.4759 - val_accuracy: 0.6750 - val_loss: 0.5005

```



3.1.6 Exercise 4 (CORE)

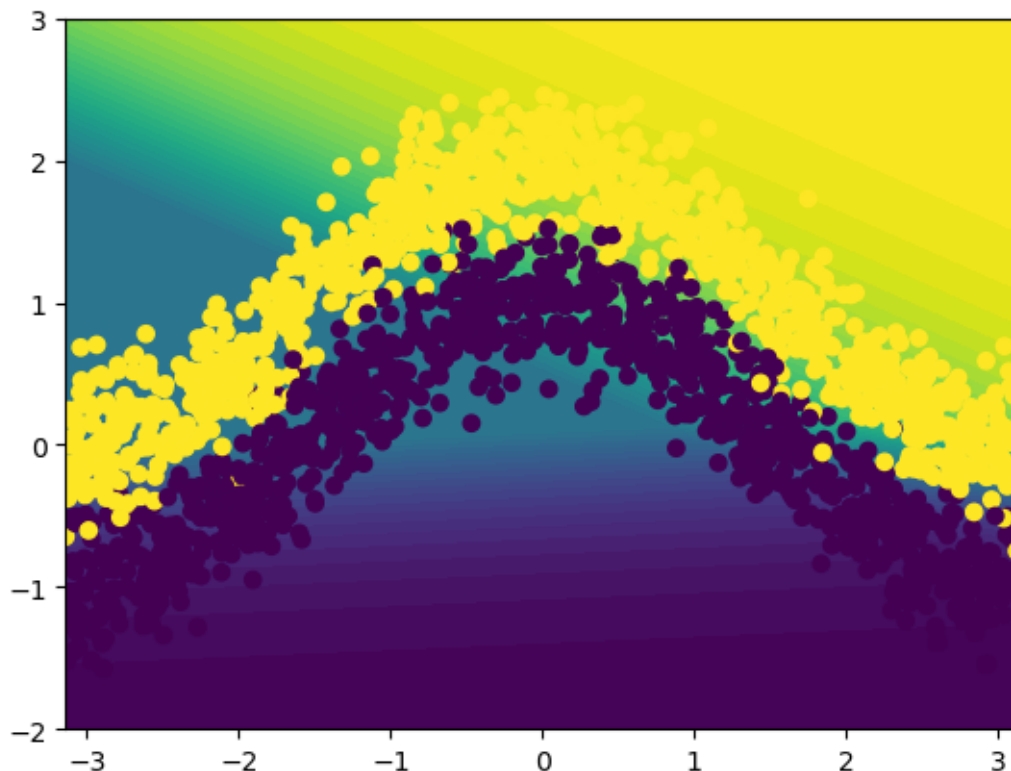
Draw a heatmap of the predicted probabilities. How does the model perform now?

```
[31]: # Code for your answer here!
x1 = np.linspace(-np.pi, np.pi, 200)
x2 = np.linspace(-2, 3, 200)
xx1, xx2 = np.meshgrid(x1, x2)
grid = np.c_[xx1.ravel(), xx2.ravel()]
probs = model_hidden2.predict(grid)
probs = probs.reshape(xx1.shape)

import matplotlib.pyplot as plt
plt.contourf(xx1, xx2, probs, levels=50)
plt.scatter(X[:,0], X[:,1], c=y)
plt.show()
```

1250/1250

1s 985us/step



the model is better because it can create nonlinear boundary.

3.1.7 Visualizing the output of each neuron

By visualizing the output of each neuron, we can gain an understanding of the features extracted.

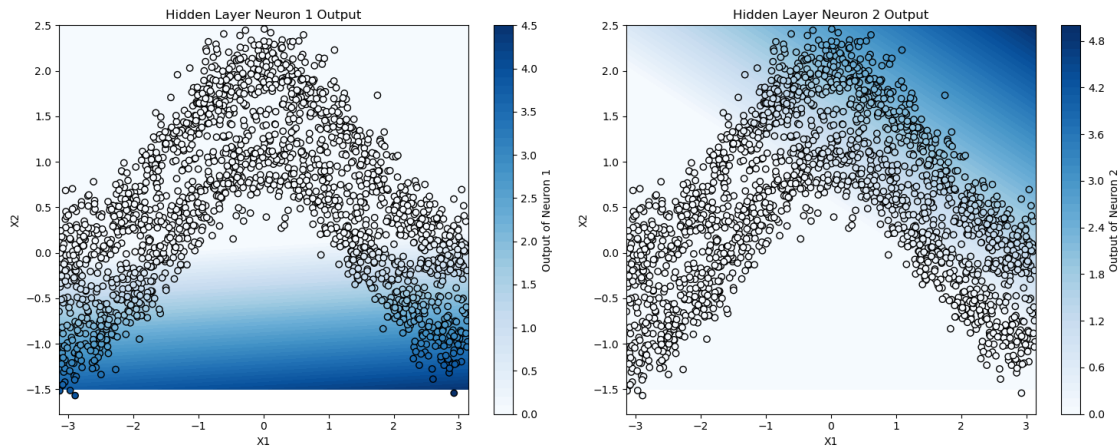
In the code below, we create a keras model for the mapping of the inputs to the hidden layer and pass through the data. By plotting the output of the hidden layer, we observe that: - the first hidden feature captures the separation of the two classes when $x_1 < 0$ and - the second hidden feature captures the separation of the two classes when $x_1 > 0$.

```
[17]: # Extract the output of the hidden layer
hidden_layer_model = keras.Model(inputs=model_hidden2.input,
    ↪outputs=model_hidden2.layers[1].output)

# Hidden output for the observed data
z =hidden_layer_model(X)
# Convert to numpy array
z = z.numpy()

# Hidden output for the test points
z_grid =hidden_layer_model(X_grid)
# Convert to numpy array
z_grid = z_grid.numpy()

[18]: # Draw a heatmap of the output of the hidden layer neurons
fig, ax = plt.subplots(1, 2, figsize=(15, 6))
for i in range(2):
    vmin= min(z[:,i].min(), z_grid[:,i].min())
    vmax= max(z[:,i].max(), z_grid[:,i].max())
    ax[i].contourf(X_grid1, X_grid2, z_grid[:,i].reshape(X_grid1.shape),
    ↪levels=50, cmap='Blues', vmin=vmin, vmax=vmax)
    ax[i].scatter(X[:,0], X[:,1], c=z[:,i], cmap='Blues',
    ↪edgecolor='k',vmin=vmin, vmax=vmax)
    ax[i].set_xlabel('X1')
    ax[i].set_ylabel('X2')
    ax[i].set_title(f'Hidden Layer Neuron {i+1} Output')
    cbar = plt.colorbar(ax[i].collections[0], ax=ax[i])
    cbar.set_label(f'Output of Neuron {i+1}')
fig.tight_layout()
plt.show()
```

We can also print out the values of the weights and bias for each layer:

```
[19]: print('Hidden layer bias:', model_hidden2.layers[1].bias.numpy())
      print('Hidden layer weights:', model_hidden2.layers[1].kernel.numpy())
      print('Output layer bias:', model_hidden2.layers[2].bias.numpy())
      print('Output layer weights:', model_hidden2.layers[2].kernel.numpy())
```

```
Hidden layer bias: [ 0.2505037 -0.7231062]
Hidden layer weights: [[ 0.11788312  0.6690574 ]
 [-2.5601854  1.4174254 ]]
Output layer bias: [-0.453516]
Output layer weights: [[-0.900881 ]
 [ 1.2416656]]
```

3.1.8 Increasing the width of the network

3.1.9 Exercise 5 (CORE)

Now, try increasing the number of hidden units to 5. - Define and compile the model. - Fit the model and train for as many epochs find are needed. - Draw a heatmap of the predicted probabilities over the grid of input values. - Draw heatmaps of the output for each hidden unit over the grid of input values.

Comment on how the predictions have changed and what features appear to be extracted by each hidden unit. Which model would you choose and why?

```
[34]: # Code for your answer here!
input_layer = keras.Input(shape=(D,))
hidden_layer = keras.layers.Dense(5, activation='relu')(input_layer)
output_layer = keras.layers.Dense(1, activation='sigmoid')(hidden_layer)

model_hidden5 = keras.Model(inputs=input_layer, outputs=output_layer)
model_hidden5.compile(
    optimizer='adam',
```

```

    loss='binary_crossentropy',
    metrics=['accuracy']
)
perm = np.random.permutation(len(X))
X_shuffled = X[perm]
y_shuffled = y[perm]

history = model_hidden5.fit(
    X_shuffled,
    y_shuffled,
    epochs=100,
    batch_size=32,
    validation_split=0.2,
    shuffle=True
)

x1 = np.linspace(-np.pi, np.pi, 200)
x2 = np.linspace(-2, 3, 200)
xx1, xx2 = np.meshgrid(x1, x2)

grid = np.c_[xx1.ravel(), xx2.ravel()]
probs = model_hidden5.predict(grid)
probs = probs.reshape(xx1.shape)
plt.contourf(xx1, xx2, probs, levels=50)
plt.scatter(X[:,0], X[:,1], c=y, edgecolors='k')
plt.show()

```

Epoch 1/100

50/50 1s 6ms/step -
accuracy: 0.6294 - loss: 0.7038 - val_accuracy: 0.6150 - val_loss: 0.6920

Epoch 2/100

50/50 1s 13ms/step -
accuracy: 0.6344 - loss: 0.6661 - val_accuracy: 0.6150 - val_loss: 0.6564

Epoch 3/100

50/50 0s 4ms/step -
accuracy: 0.6356 - loss: 0.6434 - val_accuracy: 0.6250 - val_loss: 0.6339

Epoch 4/100

50/50 0s 4ms/step -
accuracy: 0.6369 - loss: 0.6288 - val_accuracy: 0.6325 - val_loss: 0.6188

Epoch 5/100

50/50 0s 4ms/step -
accuracy: 0.6431 - loss: 0.6183 - val_accuracy: 0.6300 - val_loss: 0.6075

Epoch 6/100

50/50 0s 4ms/step -
accuracy: 0.6425 - loss: 0.6095 - val_accuracy: 0.6350 - val_loss: 0.5982

Epoch 7/100

50/50 0s 4ms/step -

accuracy: 0.6475 - loss: 0.6013 - val_accuracy: 0.6400 - val_loss: 0.5895
 Epoch 8/100
 50/50 0s 7ms/step -
 accuracy: 0.6500 - loss: 0.5934 - val_accuracy: 0.6475 - val_loss: 0.5811
 Epoch 9/100
 50/50 0s 4ms/step -
 accuracy: 0.6581 - loss: 0.5854 - val_accuracy: 0.6650 - val_loss: 0.5729
 Epoch 10/100
 50/50 0s 4ms/step -
 accuracy: 0.6675 - loss: 0.5774 - val_accuracy: 0.6775 - val_loss: 0.5648
 Epoch 11/100
 50/50 0s 5ms/step -
 accuracy: 0.6787 - loss: 0.5693 - val_accuracy: 0.6875 - val_loss: 0.5568
 Epoch 12/100
 50/50 0s 5ms/step -
 accuracy: 0.6881 - loss: 0.5611 - val_accuracy: 0.7075 - val_loss: 0.5488
 Epoch 13/100
 50/50 0s 4ms/step -
 accuracy: 0.7000 - loss: 0.5528 - val_accuracy: 0.7250 - val_loss: 0.5408
 Epoch 14/100
 50/50 0s 4ms/step -
 accuracy: 0.7094 - loss: 0.5444 - val_accuracy: 0.7375 - val_loss: 0.5326
 Epoch 15/100
 50/50 0s 4ms/step -
 accuracy: 0.7200 - loss: 0.5357 - val_accuracy: 0.7450 - val_loss: 0.5243
 Epoch 16/100
 50/50 0s 5ms/step -
 accuracy: 0.7306 - loss: 0.5268 - val_accuracy: 0.7550 - val_loss: 0.5158
 Epoch 17/100
 50/50 0s 4ms/step -
 accuracy: 0.7369 - loss: 0.5179 - val_accuracy: 0.7625 - val_loss: 0.5071
 Epoch 18/100
 50/50 0s 4ms/step -
 accuracy: 0.7469 - loss: 0.5086 - val_accuracy: 0.7675 - val_loss: 0.4983
 Epoch 19/100
 50/50 0s 4ms/step -
 accuracy: 0.7588 - loss: 0.4989 - val_accuracy: 0.7750 - val_loss: 0.4890
 Epoch 20/100
 50/50 0s 5ms/step -
 accuracy: 0.7669 - loss: 0.4888 - val_accuracy: 0.7875 - val_loss: 0.4794
 Epoch 21/100
 50/50 0s 4ms/step -
 accuracy: 0.7806 - loss: 0.4783 - val_accuracy: 0.7975 - val_loss: 0.4692
 Epoch 22/100
 50/50 0s 4ms/step -
 accuracy: 0.7919 - loss: 0.4673 - val_accuracy: 0.8050 - val_loss: 0.4585
 Epoch 23/100
 50/50 0s 4ms/step -

accuracy: 0.8031 - loss: 0.4561 - val_accuracy: 0.8175 - val_loss: 0.4477
 Epoch 24/100
 50/50 0s 3ms/step -
 accuracy: 0.8131 - loss: 0.4446 - val_accuracy: 0.8275 - val_loss: 0.4368
 Epoch 25/100
 50/50 0s 4ms/step -
 accuracy: 0.8219 - loss: 0.4331 - val_accuracy: 0.8300 - val_loss: 0.4263
 Epoch 26/100
 50/50 0s 4ms/step -
 accuracy: 0.8306 - loss: 0.4223 - val_accuracy: 0.8350 - val_loss: 0.4165
 Epoch 27/100
 50/50 0s 4ms/step -
 accuracy: 0.8356 - loss: 0.4121 - val_accuracy: 0.8375 - val_loss: 0.4069
 Epoch 28/100
 50/50 0s 4ms/step -
 accuracy: 0.8394 - loss: 0.4021 - val_accuracy: 0.8375 - val_loss: 0.3976
 Epoch 29/100
 50/50 0s 4ms/step -
 accuracy: 0.8456 - loss: 0.3924 - val_accuracy: 0.8500 - val_loss: 0.3885
 Epoch 30/100
 50/50 0s 4ms/step -
 accuracy: 0.8506 - loss: 0.3831 - val_accuracy: 0.8525 - val_loss: 0.3797
 Epoch 31/100
 50/50 0s 4ms/step -
 accuracy: 0.8575 - loss: 0.3743 - val_accuracy: 0.8600 - val_loss: 0.3713
 Epoch 32/100
 50/50 0s 4ms/step -
 accuracy: 0.8612 - loss: 0.3658 - val_accuracy: 0.8675 - val_loss: 0.3632
 Epoch 33/100
 50/50 0s 4ms/step -
 accuracy: 0.8669 - loss: 0.3578 - val_accuracy: 0.8700 - val_loss: 0.3554
 Epoch 34/100
 50/50 0s 4ms/step -
 accuracy: 0.8712 - loss: 0.3501 - val_accuracy: 0.8725 - val_loss: 0.3482
 Epoch 35/100
 50/50 0s 4ms/step -
 accuracy: 0.8763 - loss: 0.3428 - val_accuracy: 0.8775 - val_loss: 0.3412
 Epoch 36/100
 50/50 0s 4ms/step -
 accuracy: 0.8844 - loss: 0.3357 - val_accuracy: 0.8850 - val_loss: 0.3344
 Epoch 37/100
 50/50 0s 4ms/step -
 accuracy: 0.8894 - loss: 0.3290 - val_accuracy: 0.8900 - val_loss: 0.3279
 Epoch 38/100
 50/50 0s 4ms/step -
 accuracy: 0.8931 - loss: 0.3224 - val_accuracy: 0.8950 - val_loss: 0.3216
 Epoch 39/100
 50/50 0s 5ms/step -

accuracy: 0.8975 - loss: 0.3161 - val_accuracy: 0.8975 - val_loss: 0.3155
 Epoch 40/100
 50/50 0s 4ms/step -
 accuracy: 0.9019 - loss: 0.3100 - val_accuracy: 0.9050 - val_loss: 0.3095
 Epoch 41/100
 50/50 0s 4ms/step -
 accuracy: 0.9038 - loss: 0.3039 - val_accuracy: 0.9125 - val_loss: 0.3037
 Epoch 42/100
 50/50 0s 3ms/step -
 accuracy: 0.9081 - loss: 0.2980 - val_accuracy: 0.9125 - val_loss: 0.2980
 Epoch 43/100
 50/50 0s 4ms/step -
 accuracy: 0.9106 - loss: 0.2922 - val_accuracy: 0.9150 - val_loss: 0.2924
 Epoch 44/100
 50/50 0s 4ms/step -
 accuracy: 0.9125 - loss: 0.2865 - val_accuracy: 0.9150 - val_loss: 0.2871
 Epoch 45/100
 50/50 0s 3ms/step -
 accuracy: 0.9175 - loss: 0.2811 - val_accuracy: 0.9200 - val_loss: 0.2818
 Epoch 46/100
 50/50 0s 3ms/step -
 accuracy: 0.9200 - loss: 0.2757 - val_accuracy: 0.9225 - val_loss: 0.2767
 Epoch 47/100
 50/50 0s 4ms/step -
 accuracy: 0.9275 - loss: 0.2704 - val_accuracy: 0.9275 - val_loss: 0.2718
 Epoch 48/100
 50/50 0s 4ms/step -
 accuracy: 0.9325 - loss: 0.2653 - val_accuracy: 0.9300 - val_loss: 0.2670
 Epoch 49/100
 50/50 0s 4ms/step -
 accuracy: 0.9356 - loss: 0.2603 - val_accuracy: 0.9325 - val_loss: 0.2624
 Epoch 50/100
 50/50 0s 4ms/step -
 accuracy: 0.9381 - loss: 0.2555 - val_accuracy: 0.9350 - val_loss: 0.2579
 Epoch 51/100
 50/50 0s 4ms/step -
 accuracy: 0.9388 - loss: 0.2508 - val_accuracy: 0.9375 - val_loss: 0.2536
 Epoch 52/100
 50/50 0s 5ms/step -
 accuracy: 0.9406 - loss: 0.2462 - val_accuracy: 0.9375 - val_loss: 0.2493
 Epoch 53/100
 50/50 0s 4ms/step -
 accuracy: 0.9413 - loss: 0.2417 - val_accuracy: 0.9450 - val_loss: 0.2451
 Epoch 54/100
 50/50 0s 4ms/step -
 accuracy: 0.9438 - loss: 0.2374 - val_accuracy: 0.9450 - val_loss: 0.2410
 Epoch 55/100
 50/50 0s 4ms/step -

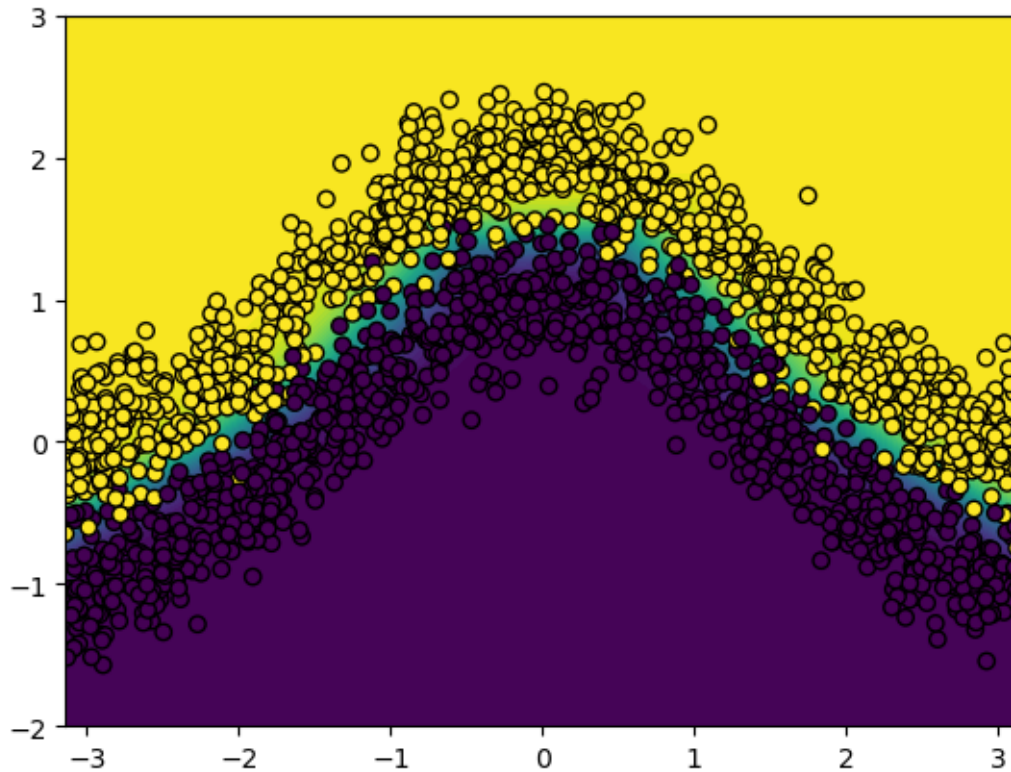
accuracy: 0.9438 - loss: 0.2331 - val_accuracy: 0.9450 - val_loss: 0.2371
 Epoch 56/100
 50/50 0s 4ms/step -
 accuracy: 0.9456 - loss: 0.2290 - val_accuracy: 0.9450 - val_loss: 0.2332
 Epoch 57/100
 50/50 0s 5ms/step -
 accuracy: 0.9469 - loss: 0.2249 - val_accuracy: 0.9475 - val_loss: 0.2294
 Epoch 58/100
 50/50 0s 4ms/step -
 accuracy: 0.9481 - loss: 0.2210 - val_accuracy: 0.9500 - val_loss: 0.2258
 Epoch 59/100
 50/50 0s 4ms/step -
 accuracy: 0.9488 - loss: 0.2171 - val_accuracy: 0.9500 - val_loss: 0.2222
 Epoch 60/100
 50/50 0s 4ms/step -
 accuracy: 0.9494 - loss: 0.2134 - val_accuracy: 0.9475 - val_loss: 0.2188
 Epoch 61/100
 50/50 0s 5ms/step -
 accuracy: 0.9513 - loss: 0.2098 - val_accuracy: 0.9525 - val_loss: 0.2154
 Epoch 62/100
 50/50 0s 3ms/step -
 accuracy: 0.9525 - loss: 0.2062 - val_accuracy: 0.9525 - val_loss: 0.2121
 Epoch 63/100
 50/50 0s 3ms/step -
 accuracy: 0.9531 - loss: 0.2028 - val_accuracy: 0.9550 - val_loss: 0.2089
 Epoch 64/100
 50/50 0s 3ms/step -
 accuracy: 0.9544 - loss: 0.1995 - val_accuracy: 0.9550 - val_loss: 0.2059
 Epoch 65/100
 50/50 0s 5ms/step -
 accuracy: 0.9550 - loss: 0.1962 - val_accuracy: 0.9550 - val_loss: 0.2029
 Epoch 66/100
 50/50 0s 3ms/step -
 accuracy: 0.9563 - loss: 0.1931 - val_accuracy: 0.9550 - val_loss: 0.2000
 Epoch 67/100
 50/50 0s 4ms/step -
 accuracy: 0.9575 - loss: 0.1901 - val_accuracy: 0.9575 - val_loss: 0.1971
 Epoch 68/100
 50/50 0s 4ms/step -
 accuracy: 0.9575 - loss: 0.1871 - val_accuracy: 0.9575 - val_loss: 0.1944
 Epoch 69/100
 50/50 0s 4ms/step -
 accuracy: 0.9575 - loss: 0.1842 - val_accuracy: 0.9550 - val_loss: 0.1918
 Epoch 70/100
 50/50 0s 5ms/step -
 accuracy: 0.9575 - loss: 0.1814 - val_accuracy: 0.9550 - val_loss: 0.1892
 Epoch 71/100
 50/50 0s 4ms/step -

accuracy: 0.9588 - loss: 0.1787 - val_accuracy: 0.9550 - val_loss: 0.1868
 Epoch 72/100
 50/50 0s 4ms/step -
 accuracy: 0.9594 - loss: 0.1761 - val_accuracy: 0.9550 - val_loss: 0.1844
 Epoch 73/100
 50/50 0s 4ms/step -
 accuracy: 0.9594 - loss: 0.1735 - val_accuracy: 0.9525 - val_loss: 0.1820
 Epoch 74/100
 50/50 0s 4ms/step -
 accuracy: 0.9600 - loss: 0.1711 - val_accuracy: 0.9525 - val_loss: 0.1798
 Epoch 75/100
 50/50 0s 4ms/step -
 accuracy: 0.9619 - loss: 0.1687 - val_accuracy: 0.9500 - val_loss: 0.1776
 Epoch 76/100
 50/50 0s 4ms/step -
 accuracy: 0.9625 - loss: 0.1664 - val_accuracy: 0.9500 - val_loss: 0.1755
 Epoch 77/100
 50/50 0s 4ms/step -
 accuracy: 0.9625 - loss: 0.1641 - val_accuracy: 0.9475 - val_loss: 0.1734
 Epoch 78/100
 50/50 0s 5ms/step -
 accuracy: 0.9613 - loss: 0.1619 - val_accuracy: 0.9450 - val_loss: 0.1714
 Epoch 79/100
 50/50 0s 5ms/step -
 accuracy: 0.9625 - loss: 0.1598 - val_accuracy: 0.9450 - val_loss: 0.1695
 Epoch 80/100
 50/50 0s 4ms/step -
 accuracy: 0.9631 - loss: 0.1577 - val_accuracy: 0.9450 - val_loss: 0.1676
 Epoch 81/100
 50/50 0s 3ms/step -
 accuracy: 0.9631 - loss: 0.1557 - val_accuracy: 0.9550 - val_loss: 0.1658
 Epoch 82/100
 50/50 0s 4ms/step -
 accuracy: 0.9631 - loss: 0.1538 - val_accuracy: 0.9525 - val_loss: 0.1640
 Epoch 83/100
 50/50 0s 6ms/step -
 accuracy: 0.9631 - loss: 0.1519 - val_accuracy: 0.9525 - val_loss: 0.1623
 Epoch 84/100
 50/50 0s 4ms/step -
 accuracy: 0.9644 - loss: 0.1501 - val_accuracy: 0.9525 - val_loss: 0.1607
 Epoch 85/100
 50/50 0s 4ms/step -
 accuracy: 0.9650 - loss: 0.1483 - val_accuracy: 0.9525 - val_loss: 0.1591
 Epoch 86/100
 50/50 0s 4ms/step -
 accuracy: 0.9663 - loss: 0.1466 - val_accuracy: 0.9500 - val_loss: 0.1575
 Epoch 87/100
 50/50 0s 5ms/step -

```

accuracy: 0.9675 - loss: 0.1449 - val_accuracy: 0.9500 - val_loss: 0.1560
Epoch 88/100
50/50          0s 4ms/step -
accuracy: 0.9675 - loss: 0.1433 - val_accuracy: 0.9500 - val_loss: 0.1546
Epoch 89/100
50/50          0s 4ms/step -
accuracy: 0.9675 - loss: 0.1417 - val_accuracy: 0.9500 - val_loss: 0.1532
Epoch 90/100
50/50          0s 4ms/step -
accuracy: 0.9681 - loss: 0.1402 - val_accuracy: 0.9500 - val_loss: 0.1518
Epoch 91/100
50/50          0s 4ms/step -
accuracy: 0.9681 - loss: 0.1387 - val_accuracy: 0.9525 - val_loss: 0.1505
Epoch 92/100
50/50          0s 5ms/step -
accuracy: 0.9681 - loss: 0.1373 - val_accuracy: 0.9525 - val_loss: 0.1492
Epoch 93/100
50/50          0s 4ms/step -
accuracy: 0.9681 - loss: 0.1359 - val_accuracy: 0.9525 - val_loss: 0.1479
Epoch 94/100
50/50          0s 4ms/step -
accuracy: 0.9688 - loss: 0.1345 - val_accuracy: 0.9525 - val_loss: 0.1467
Epoch 95/100
50/50          0s 5ms/step -
accuracy: 0.9688 - loss: 0.1332 - val_accuracy: 0.9525 - val_loss: 0.1455
Epoch 96/100
50/50          0s 5ms/step -
accuracy: 0.9681 - loss: 0.1319 - val_accuracy: 0.9525 - val_loss: 0.1444
Epoch 97/100
50/50          0s 4ms/step -
accuracy: 0.9681 - loss: 0.1306 - val_accuracy: 0.9525 - val_loss: 0.1433
Epoch 98/100
50/50          0s 4ms/step -
accuracy: 0.9681 - loss: 0.1294 - val_accuracy: 0.9525 - val_loss: 0.1422
Epoch 99/100
50/50          0s 4ms/step -
accuracy: 0.9681 - loss: 0.1282 - val_accuracy: 0.9525 - val_loss: 0.1412
Epoch 100/100
50/50          0s 5ms/step -
accuracy: 0.9681 - loss: 0.1271 - val_accuracy: 0.9525 - val_loss: 0.1402
1250/1250      2s 1ms/step

```

the model with 5 hidden units hold more pattern thus can have a better prediction.

3.1.10 Increasing the depth of the network

3.1.11 Exercise 6 (CORE)

Now, consider a model with two layers: 5 hidden units in the first layer and 2 hidden units in the second layer. - Define and compile the model. - Fit the model and train for as many epochs find are needed. - Draw a heatmap of the predicted probabilities over the grid of input values. - Draw heatmaps of the output for each hidden unit (in both layers) over the grid of input values.

Comment on how the predictions have changed and differences in the features extracted by the second layer compared to the first. Which model would you choose and why?

```
[35]: # Code for your answer here!
input_layer = keras.Input(shape=(D,))
hidden1 = keras.layers.Dense(5, activation='relu')(input_layer)
hidden2 = keras.layers.Dense(2, activation='relu')(hidden1)

output_layer = keras.layers.Dense(1, activation='sigmoid')(hidden2)
model_2layer = keras.Model(inputs=input_layer, outputs=output_layer)
model_2layer.compile(
    optimizer='adam',
```

```

        loss='binary_crossentropy',
        metrics=['accuracy']
    )
    perm = np.random.permutation(len(X))
    X_shuffled = X[perm]
    y_shuffled = y[perm]

    history = model_2layer.fit(
        X_shuffled,
        y_shuffled,
        epochs=100,
        batch_size=32,
        validation_split=0.2,
        shuffle=True
    )
    x1 = np.linspace(-np.pi, np.pi, 200)
    x2 = np.linspace(-2, 3, 200)
    xx1, xx2 = np.meshgrid(x1, x2)
    grid = np.c_[xx1.ravel(), xx2.ravel()]

    probs = model_2layer.predict(grid)
    probs = probs.reshape(xx1.shape)

    plt.contourf(xx1, xx2, probs, levels=50)
    plt.scatter(X[:,0], X[:,1], c=y)
    plt.show()
    hidden1_model = keras.Model(inputs=model_2layer.input,
                                outputs=model_2layer.layers[1].output)

    hidden1_out = hidden1_model.predict(grid)
    hidden1_out = hidden1_out.reshape(xx1.shape + (5,))
    for i in range(5):
        plt.contourf(xx1, xx2, hidden1_out[:, :, i], levels=50)
        plt.colorbar()
        plt.show()
    hidden2_model = keras.Model(inputs=model_2layer.input,
                                outputs=model_2layer.layers[2].output)

    hidden2_out = hidden2_model.predict(grid)
    hidden2_out = hidden2_out.reshape(xx1.shape + (2,))
    for i in range(2):
        plt.contourf(xx1, xx2, hidden2_out[:, :, i], levels=50)
        plt.colorbar()
        plt.show()

```

Epoch 1/100

50/50 1s 8ms/step -

accuracy: 0.5269 - loss: 0.6898 - val_accuracy: 0.5200 - val_loss: 0.6893

Epoch 2/100

50/50 0s 5ms/step -
accuracy: 0.5263 - loss: 0.6837 - val_accuracy: 0.5200 - val_loss: 0.6817
Epoch 3/100

50/50 0s 4ms/step -
accuracy: 0.5719 - loss: 0.6739 - val_accuracy: 0.6200 - val_loss: 0.6702
Epoch 4/100

50/50 0s 4ms/step -
accuracy: 0.6112 - loss: 0.6636 - val_accuracy: 0.6000 - val_loss: 0.6613
Epoch 5/100

50/50 0s 6ms/step -
accuracy: 0.6106 - loss: 0.6557 - val_accuracy: 0.5950 - val_loss: 0.6558
Epoch 6/100

50/50 0s 4ms/step -
accuracy: 0.6706 - loss: 0.6323 - val_accuracy: 0.7050 - val_loss: 0.6019
Epoch 7/100

50/50 0s 4ms/step -
accuracy: 0.7188 - loss: 0.5917 - val_accuracy: 0.7125 - val_loss: 0.5882
Epoch 8/100

50/50 0s 4ms/step -
accuracy: 0.7125 - loss: 0.5819 - val_accuracy: 0.7100 - val_loss: 0.5811
Epoch 9/100

50/50 0s 6ms/step -
accuracy: 0.7125 - loss: 0.5746 - val_accuracy: 0.7100 - val_loss: 0.5750
Epoch 10/100

50/50 0s 5ms/step -
accuracy: 0.7156 - loss: 0.5686 - val_accuracy: 0.7125 - val_loss: 0.5700
Epoch 11/100

50/50 0s 5ms/step -
accuracy: 0.7181 - loss: 0.5634 - val_accuracy: 0.7175 - val_loss: 0.5653
Epoch 12/100

50/50 0s 7ms/step -
accuracy: 0.7188 - loss: 0.5587 - val_accuracy: 0.7125 - val_loss: 0.5610
Epoch 13/100

50/50 0s 5ms/step -
accuracy: 0.7206 - loss: 0.5544 - val_accuracy: 0.7100 - val_loss: 0.5570
Epoch 14/100

50/50 0s 4ms/step -
accuracy: 0.7212 - loss: 0.5505 - val_accuracy: 0.7075 - val_loss: 0.5532
Epoch 15/100

50/50 0s 5ms/step -
accuracy: 0.7244 - loss: 0.5467 - val_accuracy: 0.7100 - val_loss: 0.5497
Epoch 16/100

50/50 0s 6ms/step -
accuracy: 0.7262 - loss: 0.5432 - val_accuracy: 0.7100 - val_loss: 0.5461
Epoch 17/100

50/50 0s 6ms/step -
accuracy: 0.7287 - loss: 0.5397 - val_accuracy: 0.7100 - val_loss: 0.5427
Epoch 18/100

50/50 0s 6ms/step -
 accuracy: 0.7325 - loss: 0.5364 - val_accuracy: 0.7175 - val_loss: 0.5393
 Epoch 19/100
 50/50 0s 6ms/step -
 accuracy: 0.7369 - loss: 0.5328 - val_accuracy: 0.7225 - val_loss: 0.5359
 Epoch 20/100
 50/50 0s 4ms/step -
 accuracy: 0.7400 - loss: 0.5291 - val_accuracy: 0.7275 - val_loss: 0.5323
 Epoch 21/100
 50/50 0s 5ms/step -
 accuracy: 0.7419 - loss: 0.5250 - val_accuracy: 0.7325 - val_loss: 0.5284
 Epoch 22/100
 50/50 0s 6ms/step -
 accuracy: 0.7475 - loss: 0.5205 - val_accuracy: 0.7350 - val_loss: 0.5238
 Epoch 23/100
 50/50 0s 4ms/step -
 accuracy: 0.7519 - loss: 0.5152 - val_accuracy: 0.7300 - val_loss: 0.5189
 Epoch 24/100
 50/50 0s 5ms/step -
 accuracy: 0.7638 - loss: 0.5090 - val_accuracy: 0.7375 - val_loss: 0.5134
 Epoch 25/100
 50/50 0s 4ms/step -
 accuracy: 0.7713 - loss: 0.5017 - val_accuracy: 0.7575 - val_loss: 0.5064
 Epoch 26/100
 50/50 0s 5ms/step -
 accuracy: 0.7794 - loss: 0.4923 - val_accuracy: 0.7575 - val_loss: 0.4985
 Epoch 27/100
 50/50 0s 5ms/step -
 accuracy: 0.7944 - loss: 0.4815 - val_accuracy: 0.7750 - val_loss: 0.4877
 Epoch 28/100
 50/50 0s 5ms/step -
 accuracy: 0.8062 - loss: 0.4689 - val_accuracy: 0.7800 - val_loss: 0.4724
 Epoch 29/100
 50/50 0s 5ms/step -
 accuracy: 0.8238 - loss: 0.4533 - val_accuracy: 0.8100 - val_loss: 0.4513
 Epoch 30/100
 50/50 0s 5ms/step -
 accuracy: 0.8487 - loss: 0.4319 - val_accuracy: 0.8450 - val_loss: 0.4232
 Epoch 31/100
 50/50 0s 4ms/step -
 accuracy: 0.8825 - loss: 0.4035 - val_accuracy: 0.8850 - val_loss: 0.3934
 Epoch 32/100
 50/50 0s 5ms/step -
 accuracy: 0.9100 - loss: 0.3758 - val_accuracy: 0.8975 - val_loss: 0.3678
 Epoch 33/100
 50/50 0s 5ms/step -
 accuracy: 0.9262 - loss: 0.3540 - val_accuracy: 0.9125 - val_loss: 0.3487
 Epoch 34/100

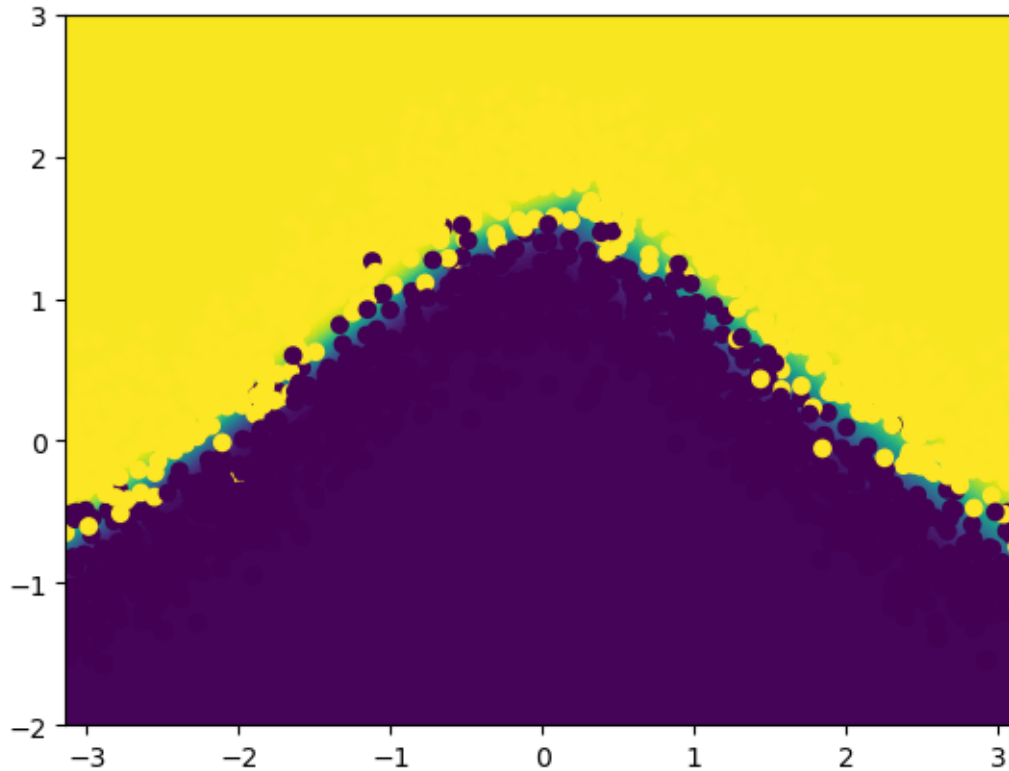
50/50 0s 5ms/step -
 accuracy: 0.9356 - loss: 0.3378 - val_accuracy: 0.9250 - val_loss: 0.3344
 Epoch 35/100
 50/50 0s 4ms/step -
 accuracy: 0.9450 - loss: 0.3243 - val_accuracy: 0.9325 - val_loss: 0.3224
 Epoch 36/100
 50/50 0s 4ms/step -
 accuracy: 0.9506 - loss: 0.3123 - val_accuracy: 0.9325 - val_loss: 0.3117
 Epoch 37/100
 50/50 0s 5ms/step -
 accuracy: 0.9519 - loss: 0.3015 - val_accuracy: 0.9375 - val_loss: 0.3018
 Epoch 38/100
 50/50 0s 5ms/step -
 accuracy: 0.9538 - loss: 0.2915 - val_accuracy: 0.9375 - val_loss: 0.2925
 Epoch 39/100
 50/50 0s 4ms/step -
 accuracy: 0.9556 - loss: 0.2822 - val_accuracy: 0.9400 - val_loss: 0.2837
 Epoch 40/100
 50/50 0s 7ms/step -
 accuracy: 0.9569 - loss: 0.2735 - val_accuracy: 0.9450 - val_loss: 0.2756
 Epoch 41/100
 50/50 0s 4ms/step -
 accuracy: 0.9588 - loss: 0.2654 - val_accuracy: 0.9500 - val_loss: 0.2681
 Epoch 42/100
 50/50 0s 5ms/step -
 accuracy: 0.9588 - loss: 0.2579 - val_accuracy: 0.9525 - val_loss: 0.2608
 Epoch 43/100
 50/50 0s 4ms/step -
 accuracy: 0.9600 - loss: 0.2508 - val_accuracy: 0.9550 - val_loss: 0.2540
 Epoch 44/100
 50/50 0s 5ms/step -
 accuracy: 0.9613 - loss: 0.2441 - val_accuracy: 0.9575 - val_loss: 0.2477
 Epoch 45/100
 50/50 0s 6ms/step -
 accuracy: 0.9619 - loss: 0.2377 - val_accuracy: 0.9575 - val_loss: 0.2416
 Epoch 46/100
 50/50 0s 4ms/step -
 accuracy: 0.9619 - loss: 0.2317 - val_accuracy: 0.9550 - val_loss: 0.2359
 Epoch 47/100
 50/50 0s 6ms/step -
 accuracy: 0.9625 - loss: 0.2260 - val_accuracy: 0.9550 - val_loss: 0.2305
 Epoch 48/100
 50/50 0s 5ms/step -
 accuracy: 0.9625 - loss: 0.2207 - val_accuracy: 0.9575 - val_loss: 0.2254
 Epoch 49/100
 50/50 0s 5ms/step -
 accuracy: 0.9638 - loss: 0.2156 - val_accuracy: 0.9575 - val_loss: 0.2205
 Epoch 50/100

50/50 0s 4ms/step -
 accuracy: 0.9644 - loss: 0.2108 - val_accuracy: 0.9575 - val_loss: 0.2159
 Epoch 51/100
 50/50 0s 4ms/step -
 accuracy: 0.9638 - loss: 0.2063 - val_accuracy: 0.9575 - val_loss: 0.2115
 Epoch 52/100
 50/50 0s 5ms/step -
 accuracy: 0.9644 - loss: 0.2019 - val_accuracy: 0.9550 - val_loss: 0.2073
 Epoch 53/100
 50/50 0s 4ms/step -
 accuracy: 0.9650 - loss: 0.1978 - val_accuracy: 0.9575 - val_loss: 0.2033
 Epoch 54/100
 50/50 0s 3ms/step -
 accuracy: 0.9650 - loss: 0.1938 - val_accuracy: 0.9575 - val_loss: 0.1995
 Epoch 55/100
 50/50 0s 4ms/step -
 accuracy: 0.9650 - loss: 0.1901 - val_accuracy: 0.9575 - val_loss: 0.1959
 Epoch 56/100
 50/50 0s 4ms/step -
 accuracy: 0.9656 - loss: 0.1865 - val_accuracy: 0.9575 - val_loss: 0.1924
 Epoch 57/100
 50/50 0s 4ms/step -
 accuracy: 0.9663 - loss: 0.1831 - val_accuracy: 0.9575 - val_loss: 0.1892
 Epoch 58/100
 50/50 0s 4ms/step -
 accuracy: 0.9663 - loss: 0.1799 - val_accuracy: 0.9575 - val_loss: 0.1861
 Epoch 59/100
 50/50 0s 3ms/step -
 accuracy: 0.9663 - loss: 0.1768 - val_accuracy: 0.9600 - val_loss: 0.1831
 Epoch 60/100
 50/50 0s 3ms/step -
 accuracy: 0.9663 - loss: 0.1738 - val_accuracy: 0.9600 - val_loss: 0.1802
 Epoch 61/100
 50/50 0s 3ms/step -
 accuracy: 0.9663 - loss: 0.1710 - val_accuracy: 0.9625 - val_loss: 0.1775
 Epoch 62/100
 50/50 0s 3ms/step -
 accuracy: 0.9663 - loss: 0.1683 - val_accuracy: 0.9625 - val_loss: 0.1749
 Epoch 63/100
 50/50 0s 3ms/step -
 accuracy: 0.9663 - loss: 0.1657 - val_accuracy: 0.9625 - val_loss: 0.1724
 Epoch 64/100
 50/50 0s 3ms/step -
 accuracy: 0.9663 - loss: 0.1632 - val_accuracy: 0.9625 - val_loss: 0.1700
 Epoch 65/100
 50/50 0s 3ms/step -
 accuracy: 0.9663 - loss: 0.1608 - val_accuracy: 0.9625 - val_loss: 0.1678
 Epoch 66/100

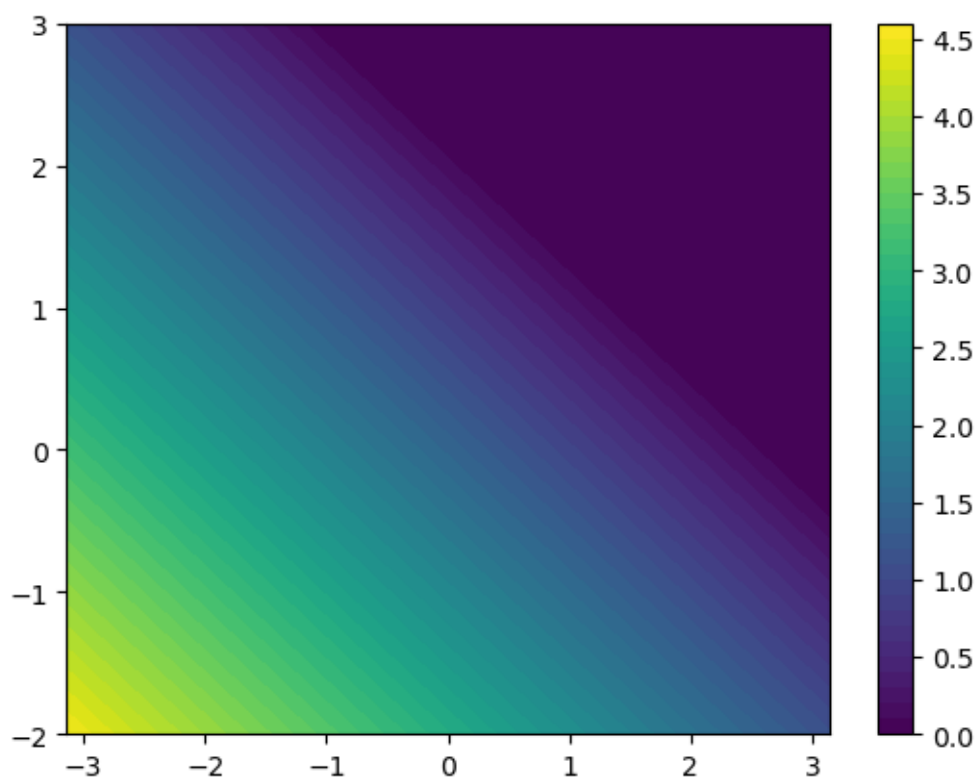
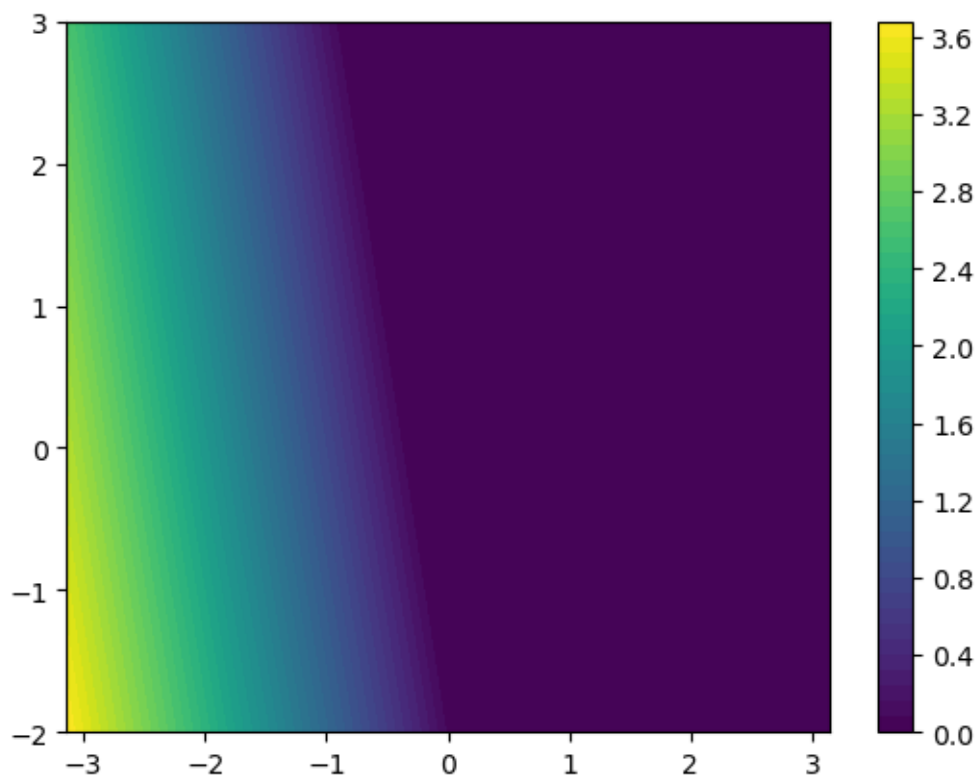
50/50 0s 5ms/step -
 accuracy: 0.9663 - loss: 0.1585 - val_accuracy: 0.9625 - val_loss: 0.1656
 Epoch 67/100
 50/50 0s 4ms/step -
 accuracy: 0.9669 - loss: 0.1563 - val_accuracy: 0.9625 - val_loss: 0.1634
 Epoch 68/100
 50/50 0s 4ms/step -
 accuracy: 0.9669 - loss: 0.1541 - val_accuracy: 0.9625 - val_loss: 0.1615
 Epoch 69/100
 50/50 0s 3ms/step -
 accuracy: 0.9669 - loss: 0.1521 - val_accuracy: 0.9625 - val_loss: 0.1595
 Epoch 70/100
 50/50 0s 4ms/step -
 accuracy: 0.9669 - loss: 0.1502 - val_accuracy: 0.9625 - val_loss: 0.1576
 Epoch 71/100
 50/50 0s 4ms/step -
 accuracy: 0.9663 - loss: 0.1483 - val_accuracy: 0.9600 - val_loss: 0.1558
 Epoch 72/100
 50/50 0s 4ms/step -
 accuracy: 0.9663 - loss: 0.1464 - val_accuracy: 0.9600 - val_loss: 0.1541
 Epoch 73/100
 50/50 0s 4ms/step -
 accuracy: 0.9663 - loss: 0.1447 - val_accuracy: 0.9600 - val_loss: 0.1524
 Epoch 74/100
 50/50 0s 3ms/step -
 accuracy: 0.9656 - loss: 0.1430 - val_accuracy: 0.9600 - val_loss: 0.1508
 Epoch 75/100
 50/50 0s 4ms/step -
 accuracy: 0.9656 - loss: 0.1414 - val_accuracy: 0.9600 - val_loss: 0.1492
 Epoch 76/100
 50/50 0s 4ms/step -
 accuracy: 0.9663 - loss: 0.1398 - val_accuracy: 0.9600 - val_loss: 0.1477
 Epoch 77/100
 50/50 0s 4ms/step -
 accuracy: 0.9663 - loss: 0.1383 - val_accuracy: 0.9600 - val_loss: 0.1463
 Epoch 78/100
 50/50 0s 4ms/step -
 accuracy: 0.9663 - loss: 0.1368 - val_accuracy: 0.9600 - val_loss: 0.1449
 Epoch 79/100
 50/50 0s 4ms/step -
 accuracy: 0.9663 - loss: 0.1354 - val_accuracy: 0.9575 - val_loss: 0.1436
 Epoch 80/100
 50/50 0s 4ms/step -
 accuracy: 0.9663 - loss: 0.1341 - val_accuracy: 0.9575 - val_loss: 0.1423
 Epoch 81/100
 50/50 0s 4ms/step -
 accuracy: 0.9663 - loss: 0.1327 - val_accuracy: 0.9575 - val_loss: 0.1410
 Epoch 82/100

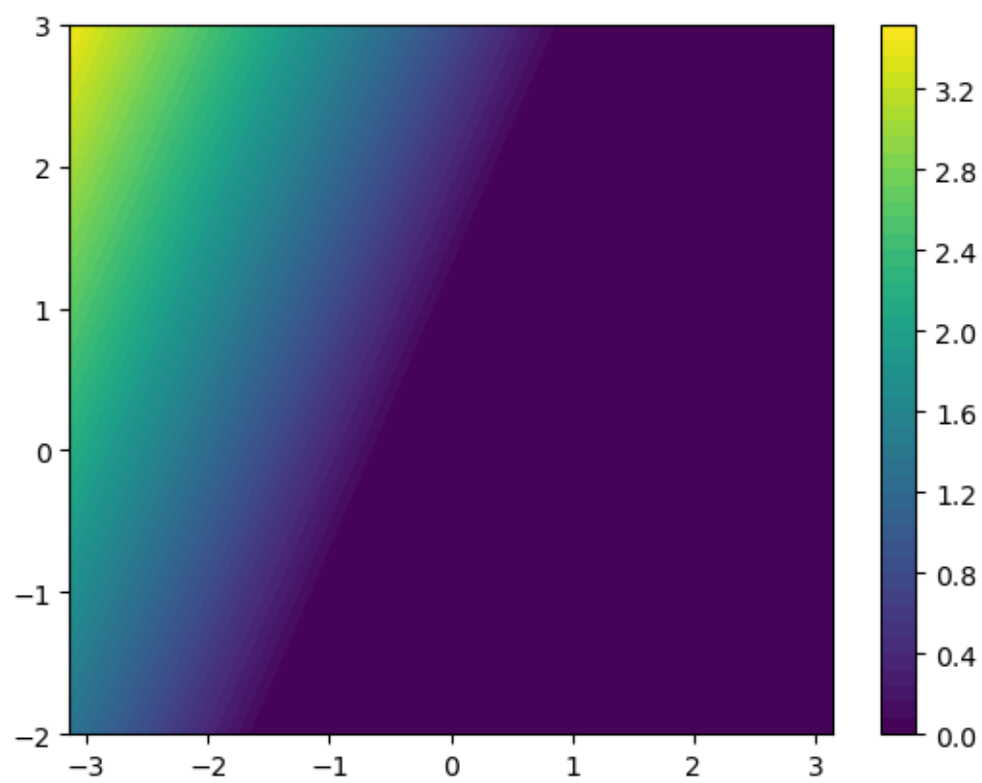
50/50 0s 4ms/step -
 accuracy: 0.9663 - loss: 0.1315 - val_accuracy: 0.9575 - val_loss: 0.1398
 Epoch 83/100
 50/50 0s 3ms/step -
 accuracy: 0.9656 - loss: 0.1303 - val_accuracy: 0.9575 - val_loss: 0.1387
 Epoch 84/100
 50/50 0s 3ms/step -
 accuracy: 0.9650 - loss: 0.1291 - val_accuracy: 0.9575 - val_loss: 0.1376
 Epoch 85/100
 50/50 0s 4ms/step -
 accuracy: 0.9656 - loss: 0.1279 - val_accuracy: 0.9550 - val_loss: 0.1365
 Epoch 86/100
 50/50 0s 5ms/step -
 accuracy: 0.9656 - loss: 0.1268 - val_accuracy: 0.9550 - val_loss: 0.1354
 Epoch 87/100
 50/50 0s 4ms/step -
 accuracy: 0.9656 - loss: 0.1257 - val_accuracy: 0.9550 - val_loss: 0.1344
 Epoch 88/100
 50/50 0s 4ms/step -
 accuracy: 0.9663 - loss: 0.1247 - val_accuracy: 0.9550 - val_loss: 0.1334
 Epoch 89/100
 50/50 0s 4ms/step -
 accuracy: 0.9663 - loss: 0.1237 - val_accuracy: 0.9550 - val_loss: 0.1324
 Epoch 90/100
 50/50 0s 4ms/step -
 accuracy: 0.9656 - loss: 0.1227 - val_accuracy: 0.9550 - val_loss: 0.1315
 Epoch 91/100
 50/50 0s 4ms/step -
 accuracy: 0.9656 - loss: 0.1218 - val_accuracy: 0.9550 - val_loss: 0.1306
 Epoch 92/100
 50/50 0s 3ms/step -
 accuracy: 0.9656 - loss: 0.1209 - val_accuracy: 0.9550 - val_loss: 0.1297
 Epoch 93/100
 50/50 0s 3ms/step -
 accuracy: 0.9656 - loss: 0.1200 - val_accuracy: 0.9550 - val_loss: 0.1289
 Epoch 94/100
 50/50 0s 3ms/step -
 accuracy: 0.9656 - loss: 0.1191 - val_accuracy: 0.9550 - val_loss: 0.1281
 Epoch 95/100
 50/50 0s 4ms/step -
 accuracy: 0.9656 - loss: 0.1183 - val_accuracy: 0.9550 - val_loss: 0.1273
 Epoch 96/100
 50/50 0s 4ms/step -
 accuracy: 0.9656 - loss: 0.1175 - val_accuracy: 0.9550 - val_loss: 0.1266
 Epoch 97/100
 50/50 0s 3ms/step -
 accuracy: 0.9656 - loss: 0.1168 - val_accuracy: 0.9550 - val_loss: 0.1258
 Epoch 98/100

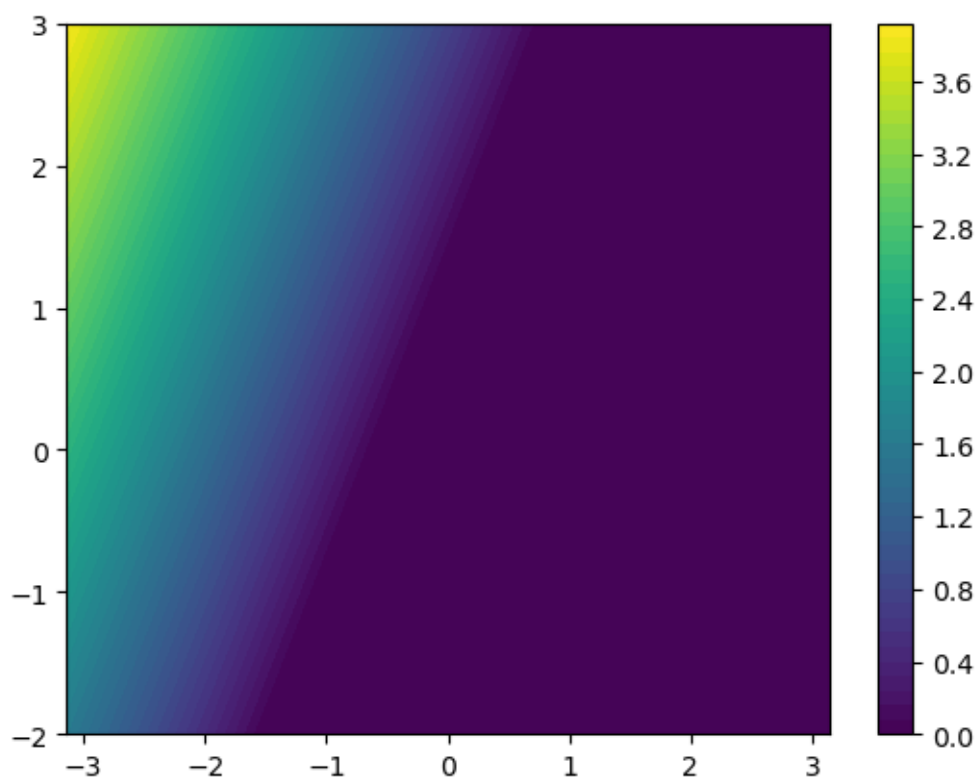
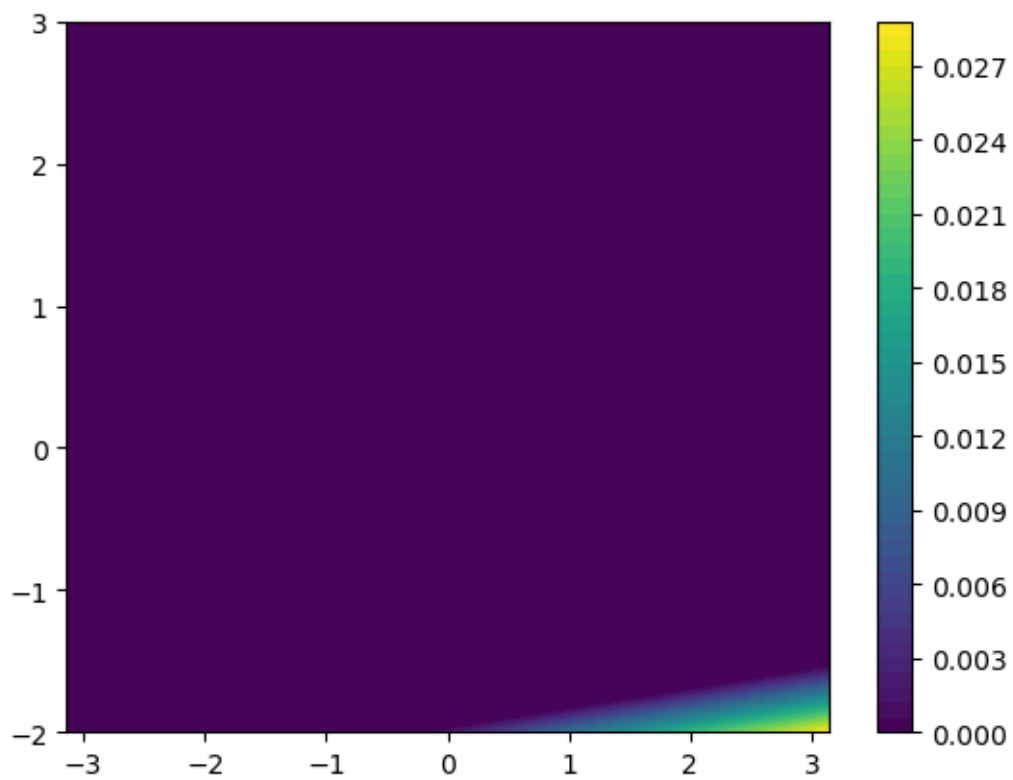

```
50/50          0s 3ms/step -  
accuracy: 0.9656 - loss: 0.1160 - val_accuracy: 0.9550 - val_loss: 0.1251  
Epoch 99/100  
50/50          0s 3ms/step -  
accuracy: 0.9656 - loss: 0.1153 - val_accuracy: 0.9550 - val_loss: 0.1244  
Epoch 100/100  
50/50          0s 3ms/step -  
accuracy: 0.9656 - loss: 0.1146 - val_accuracy: 0.9550 - val_loss: 0.1237  
1250/1250      1s 963us/step
```



```
1250/1250      1s 842us/step
```

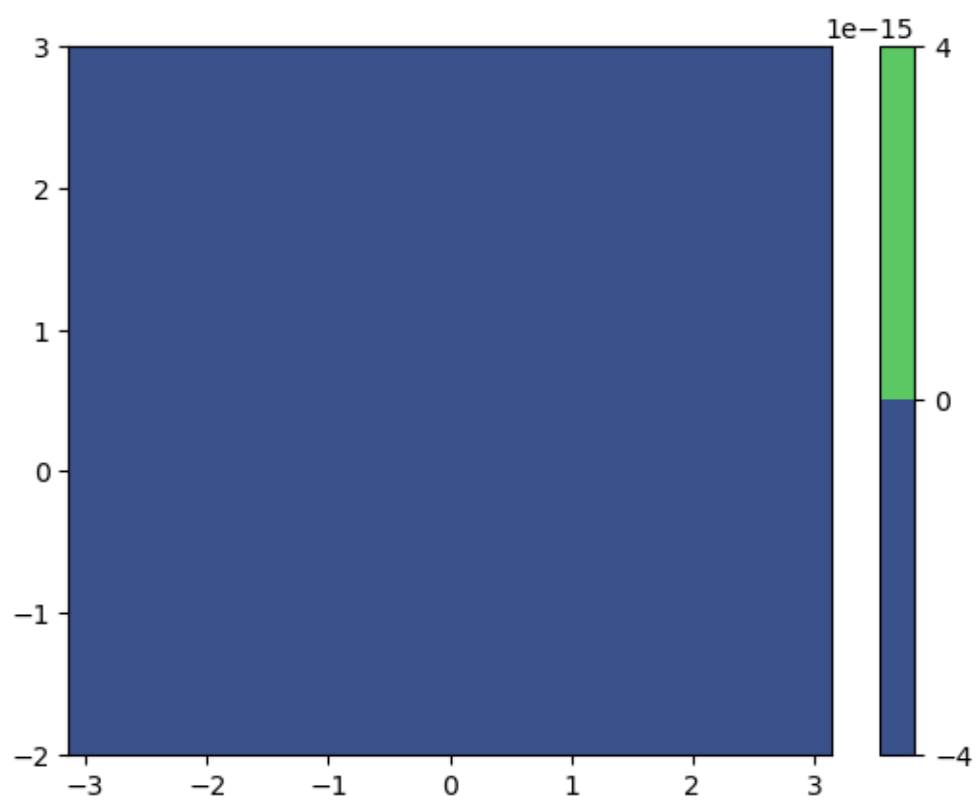


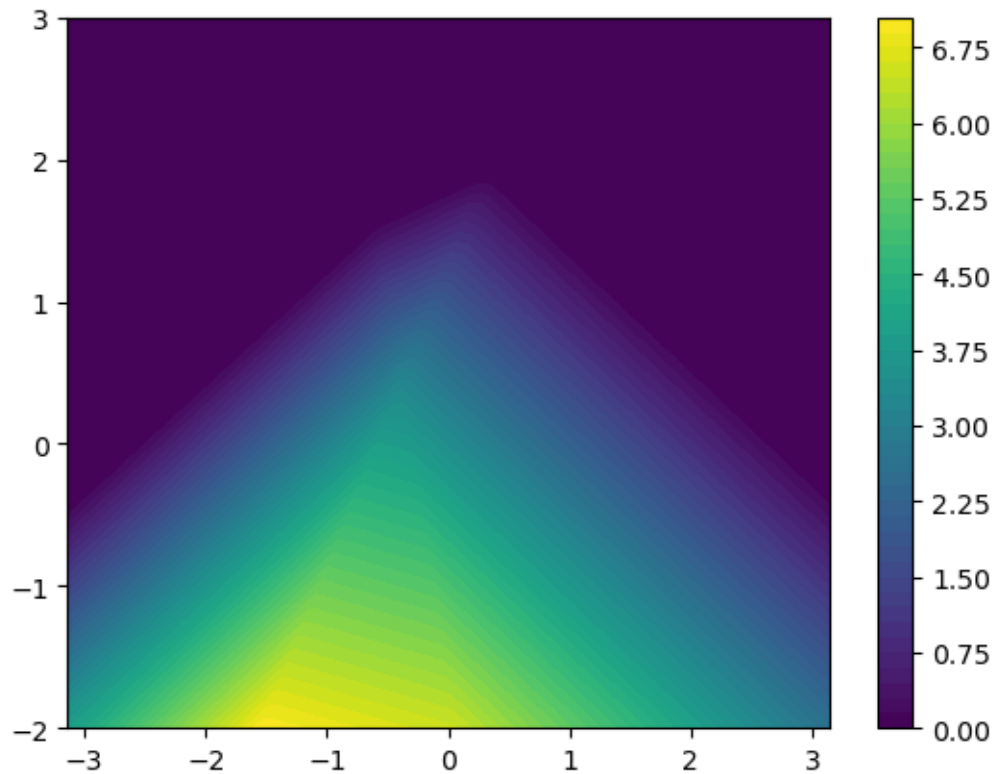




1250/1250

1s 1ms/step





i chose 2-layer model for it hold more pattern can have more precise prediction.

3.1.12 Changing the activation function

3.1.13 Exercise 7 (EXTRA)

Next, let's explore changing the activation function.

- Define and compile with your choice of width and depth (number of layers and number of hidden units within each layer), but consider a different activation function for the hidden layer, namely `tanh`.
- Fit the model and train for as many epochs find are needed.
- Draw a heatmap of the predicted probabilities over the grid of input values.
- Draw heatmaps of the output for each hidden unit over the grid of input values.

Comment on how the predictions have changed and differences in the features extracted when using the `tanh` activation function instead of the `relu` activation.

[22]: `# Code for your answer here!`

Type your answer here

4 Part 2: MNIST Data

4.0.1 Loading, exploring, and preparing the data

For the second part, we are going to use the MNIST dataset, partly because you are already familiar with it, and partly because it comes with tensorflow, the most widely used library for neural networks in python.

Let's start by load the MNIST data using `keras.datasets.mnist.load_data()`

```
[23]: (X_train, y_train), (X_test, y_test) = keras.datasets.mnist.load_data()

print("Training data shape:", X_train.shape)
print("Training labels shape:", y_train.shape)
print("Testing data shape:", X_test.shape)
print("Testing labels shape:", y_test.shape)
```

Training data shape: (60000, 28, 28)

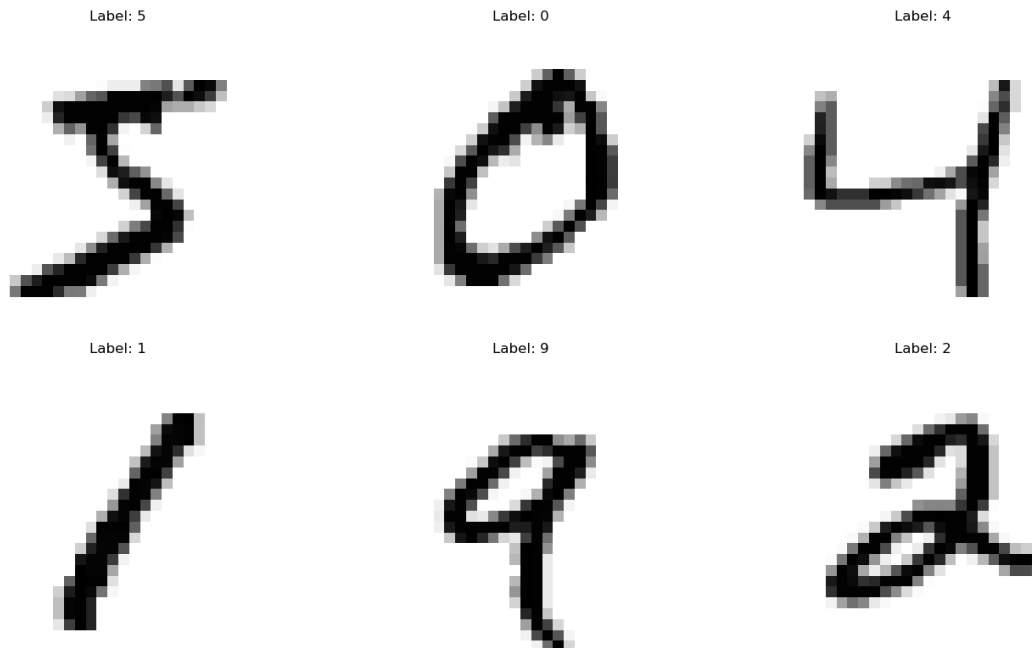
Training labels shape: (60000,)

Testing data shape: (10000, 28, 28)

Testing labels shape: (10000,)

Let's display the six images in the training set.

```
[24]: fig, ax = plt.subplots(2, 3, figsize=(15, 8))
ax = ax.flatten()
for i in range(6):
    ax[i].imshow(X_train[i], cmap="gray_r")
    ax[i].set_title(f"Label: {y_train[i]}")
    ax[i].axis('off')
fig.tight_layout()
plt.show()
```



Next, we divide the features by 255 (the maximum value of the pixels), so that all pixel values in the training and testing data are between 0 and 1.

In the workshop, we will be working only with fully connected feed-forward networks (that don't exploit the spatial structure of the images like convolutional neural networks). As such, we also flatten all images to vectors.

```
[25]: # Rescale the data
X_train = X_train / 255.0
X_test = X_test / 255.0

# Reshape the data
X_train = X_train.reshape((-1, 28 * 28))
X_test = X_test.reshape((-1, 28 * 28))
```

Lastly, we use convert the labels using a `LabelEncoder`.

```
[26]: # Converting the labels
from sklearn.preprocessing import LabelEncoder
label_encoder = LabelEncoder()
y_train = label_encoder.fit_transform(y_train)
y_test = label_encoder.transform(y_test)
```

4.0.2 Exercise 8 (CORE)

As a baseline model, create a pipeline (called `baseline`) that - first uses PCA to reduce the dimension to 100 - then uses logistic regression with no penalty

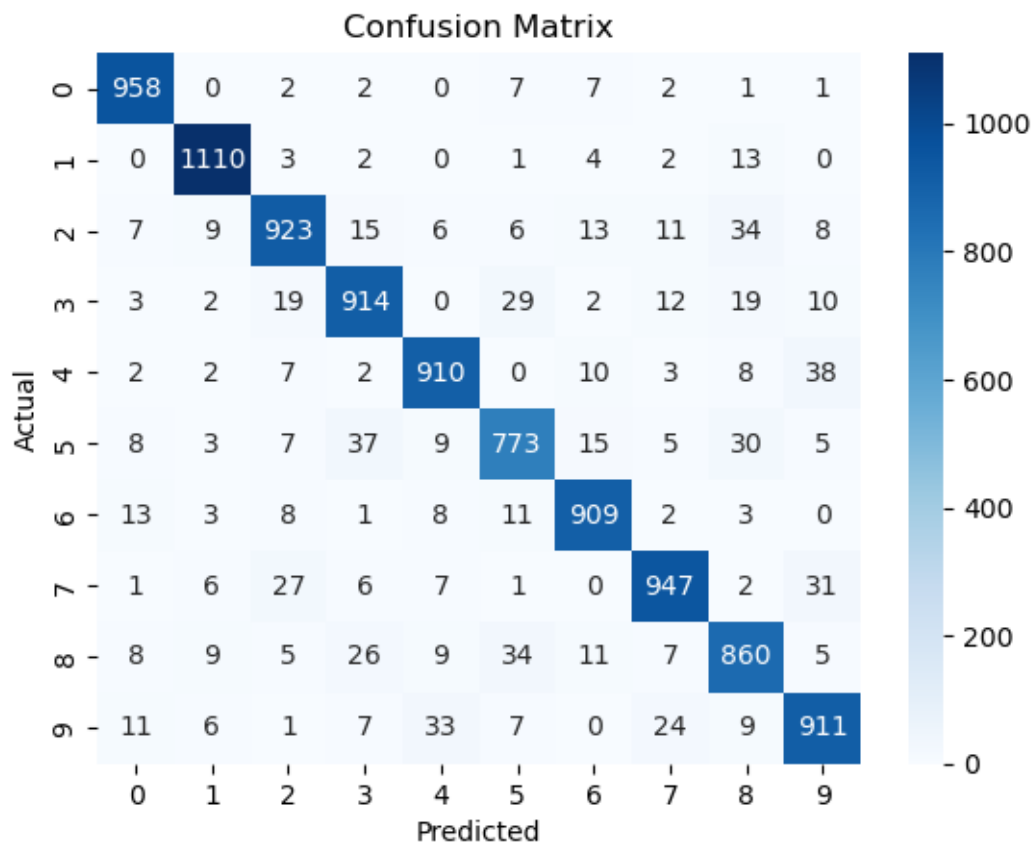
Fit the pipeline to the data. Plot the confusion matrix and print the classification report. How many parameters does the logistic regression model have?

```
[ ]: from sklearn.pipeline import make_pipeline
      from sklearn.decomposition import PCA
      from sklearn.linear_model import LogisticRegression
      from sklearn.metrics import ConfusionMatrixDisplay
      from sklearn.metrics import classification_report
      from sklearn.metrics import log_loss
      from sklearn.pipeline import Pipeline
      # Code for your answer here
      baseline = Pipeline([
          ("pca", PCA(n_components=100)),
          ("lr", LogisticRegression(penalty=None, max_iter=1000))
      ])
      baseline.fit(X_train, y_train)
      y_test_pred = baseline.predict(X_test)
      cm = confusion_matrix(y_test, y_pred)

      sns.heatmap(cm, annot=True, fmt='d', cmap='Blues')
      plt.xlabel("Predicted")
      plt.ylabel("Actual")
      plt.title("Confusion Matrix")
      plt.show()

      print(classification_report(y_test, y_pred))
```

```
/opt/conda/lib/python3.12/site-packages/sklearn/linear_model/_logistic.py:1135:
FutureWarning: 'penalty' was deprecated in version 1.8 and will be removed in
1.10. To avoid this warning, leave 'penalty' set to its default value and use
'l1_ratio' or 'C' instead. Use l1_ratio=0 instead of penalty='l2', l1_ratio=1
instead of penalty='l1', and C=np.inf instead of penalty=None.
      warnings.warn(
```



	precision	recall	f1-score	support
0	0.95	0.98	0.96	980
1	0.97	0.98	0.97	1135
2	0.92	0.89	0.91	1032
3	0.90	0.90	0.90	1010
4	0.93	0.93	0.93	982
5	0.89	0.87	0.88	892
6	0.94	0.95	0.94	958
7	0.93	0.92	0.93	1028
8	0.88	0.88	0.88	974
9	0.90	0.90	0.90	1009
accuracy			0.92	10000
macro avg	0.92	0.92	0.92	10000
weighted avg	0.92	0.92	0.92	10000

Let's plot some of the images that were both correctly and incorrectly classified, along with the corresponding probabilities.

Note: if you have called your model something other than **baseline**, please change the name in the code below.

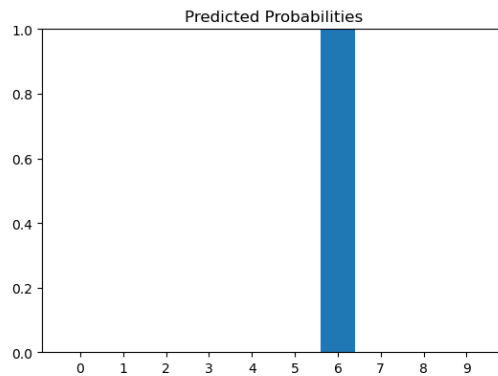
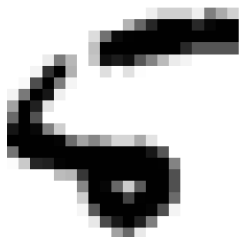
```
[40]: # indices of misclassified examples
misclas_indices = np.where(y_test != y_test_pred)[0]
# indices of correctly classified examples
clas_indices = np.where(y_test == y_test_pred)[0]

# For the first 5 misclassified examples, plot the image and a bar plot of the
# predicted probabilities for each class
fig, ax = plt.subplots(5, 2, figsize=(10, 20))
for i in range(5):
    idx = misclas_indices[i]
    ax[i, 0].imshow(X_test[idx].reshape(28, 28), cmap='gray_r')
    ax[i, 0].set_title(f'True label: {y_test[idx]}, Predicted: {y_test_pred[idx]}')
    ax[i, 0].axis('off')

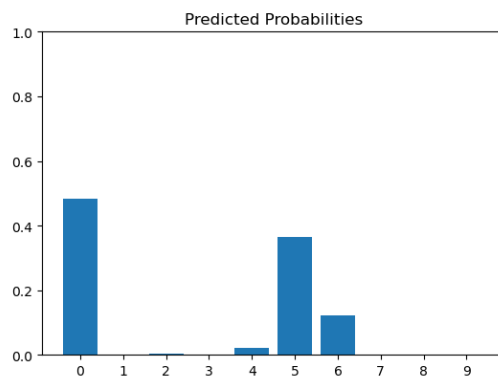
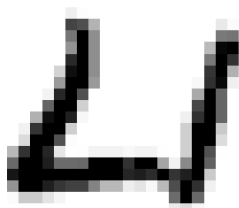
    # Get predicted probabilities for each class
    y_prob = baseline.predict_proba(X_test[idx].reshape(1, -1))[0]

    # Bar plot of predicted probabilities
    ax[i, 1].bar(range(10), y_prob)
    ax[i, 1].set_xticks(range(10))
    ax[i, 1].set_xticklabels(label_encoder.classes_)
    ax[i, 1].set_ylim(0, 1)
    ax[i, 1].set_title('Predicted Probabilities')
fig.tight_layout()
plt.show()
```

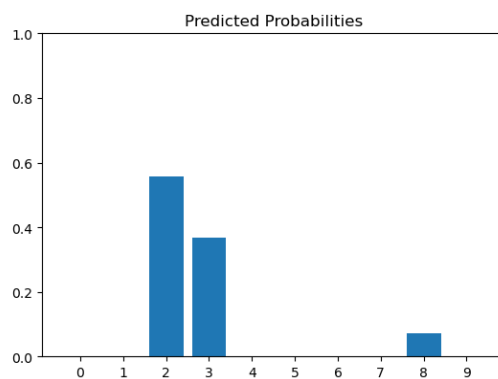
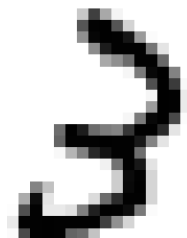
True label: 5, Predicted: 6



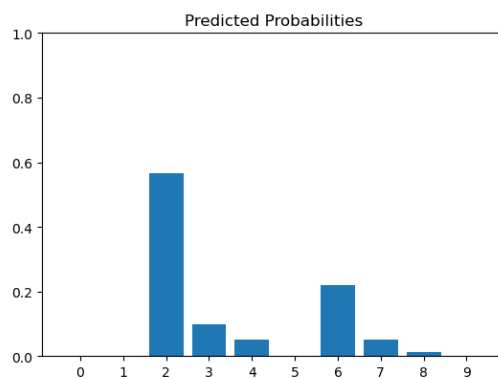
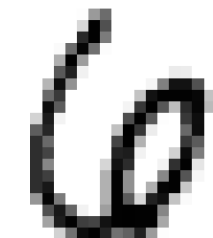
True label: 4, Predicted: 0



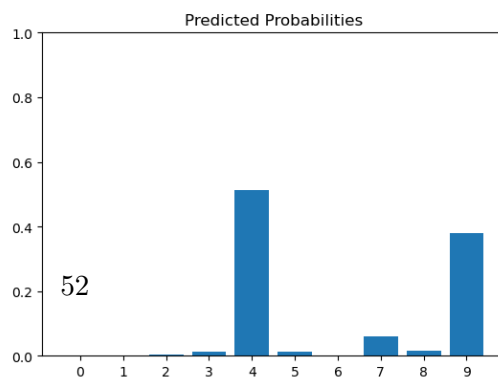
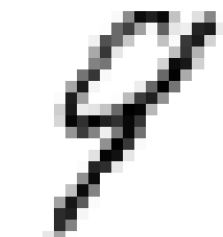
True label: 3, Predicted: 2



True label: 6, Predicted: 2



True label: 9, Predicted: 4



```

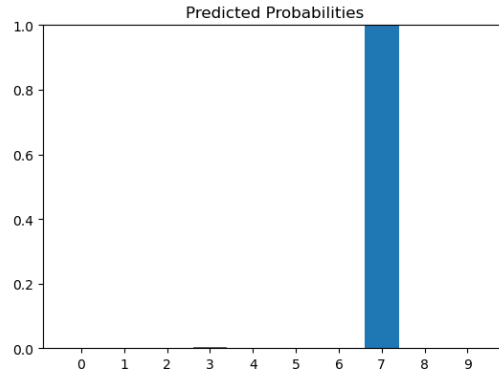
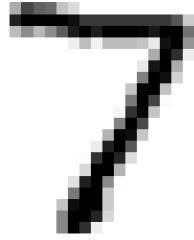
[41]: # For the first 5 correctly classified examples, plot the image and a bar plot
      ↪ of the predicted probabilities for each class
fig, ax = plt.subplots(5, 2, figsize=(10, 20))
for i in range(5):
    idx = clas_indices[i]
    ax[i, 0].imshow(X_test[idx].reshape(28, 28), cmap='gray_r')
    ax[i, 0].set_title(f'True label: {y_test[idx]}, Predicted:
    ↪ {y_test_pred[idx]}')
    ax[i, 0].axis('off')

    # Get predicted probabilities for each class
    y_prob = baseline.predict_proba(X_test[idx].reshape(1, -1))[0]

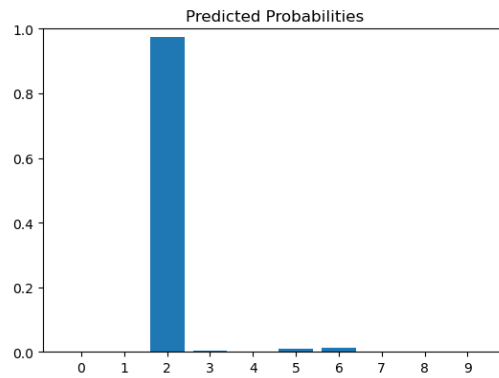
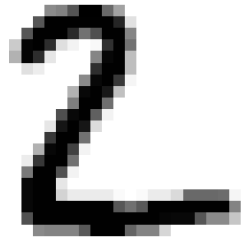
    # Bar plot of predicted probabilities
    ax[i, 1].bar(range(10), y_prob)
    ax[i, 1].set_xticks(range(10))
    ax[i, 1].set_xticklabels(label_encoder.classes_)
    ax[i, 1].set_ylim(0, 1)
    ax[i, 1].set_title('Predicted Probabilities')
fig.tight_layout()
plt.show()

```

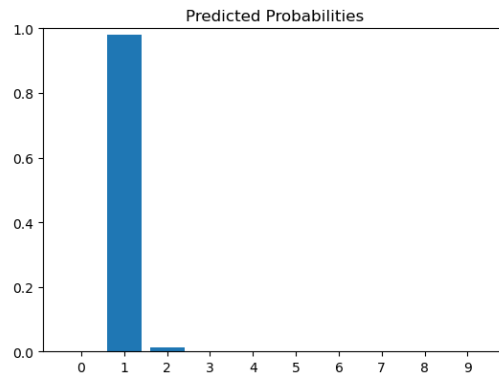
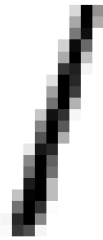
True label: 7, Predicted: 7



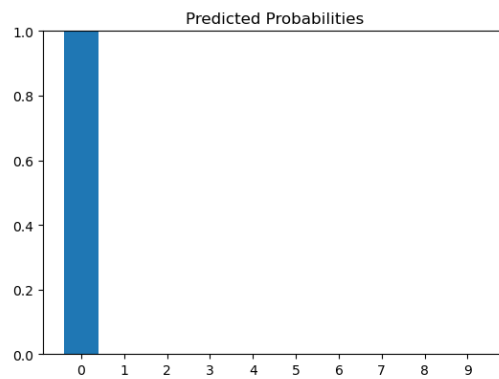
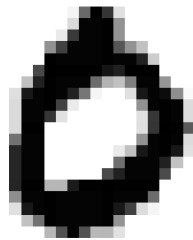
True label: 2, Predicted: 2



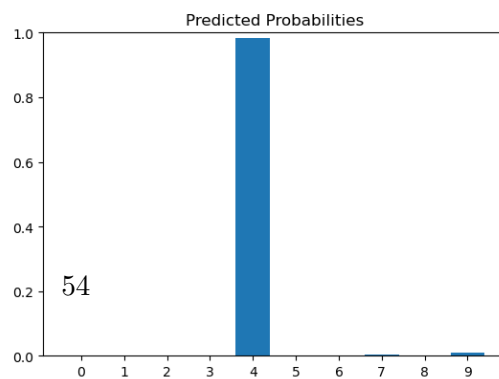
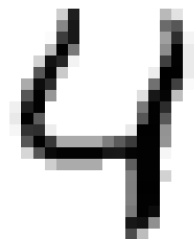
True label: 1, Predicted: 1



True label: 0, Predicted: 0



True label: 4, Predicted: 4



It can be concerning when we have high probability/confidence for incorrectly classified images. This is quantified across all data points by the cross entropy loss, which for a single data points computes:

$$L(y, p) = \sum_{c=1}^C y_c \log(p_c).$$

Notice that if the probability of the true class is close to 1, then the loss is zero, but if it is close to zero that the loss is high.

We can report the cross entropy loss, averaged across all data points.

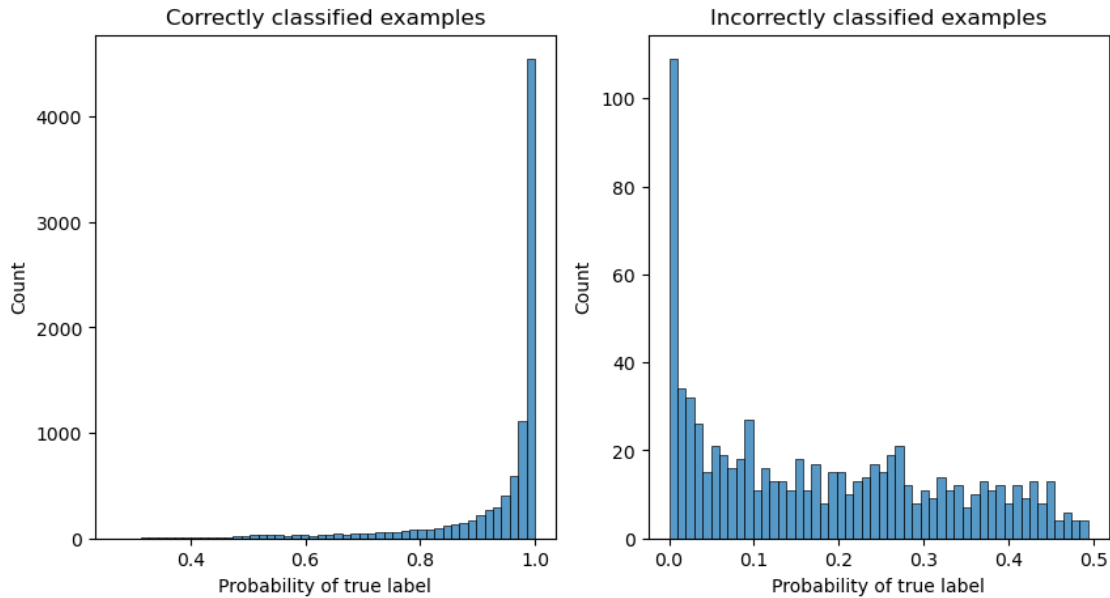
In addition, through histogram showing the distribution of the probability of that y is equal to the true test label, we can gain a more detailed understanding. We see that for the correctly classified images, over 4,000 images are correctly classified with a high probability. On the other hand, there are over 100 images that are misclassified with a high probability.

```
[42]: y_test_pred_prob = baseline.predict_proba(X_test)
print('Cross entropy loss:', log_loss(y_test, y_test_pred_prob,
    ↪ labels=label_encoder.classes_))

# Extract probability of correct label for each data point
prob_correct = y_test_pred_prob[np.arange(len(y_test)), y_test]

# Plot a histogram of the probabilities across data points, for correct or
    ↪ incorrect classification
fig, ax = plt.subplots(1, 2, figsize = (10, 5))
sns.histplot(x = prob_correct[clas_indices], bins=50, ax=ax[0])
ax[0].set_title('Correctly classified examples')
ax[0].set_xlabel('Probability of true label')
sns.histplot(x = prob_correct[misclas_indices], bins=50, ax=ax[1])
ax[1].set_xlabel('Probability of true label')
ax[1].set_title('Incorrectly classified examples')
plt.show()
```

Cross entropy loss: 0.28039545655373355



4.0.3 Exercise 9 (CORE)

Next, let's fit a fully-connected feed forward neural network. Note that we need to one-hot encoding the output labels for use in keras in the multiclass setting.

- Define and compile a model with one hidden layer of 128 neurons and ReLU activation. For the final layer, use a softmax with 10 units to obtain the probabilities for each class. Here use the categorical cross entropy loss.
- Fit the model and train for 10 epochs with a batch size of 128 and a 10% validation split, and plot the loss and accuracy for the training and validation sets as a function of epochs.
- Plot the confusion matrix and print the classification report on the test data.
- Visualize some the test images, along with their probabilities for both correctly and incorrectly classified images.
- Print the cross entropy loss on the test data, and visualize the distribution of the probability of the true labels for correctly and incorrectly classified test points.

```
[ ]: # First create a one-hot encoding of the labels for use in Keras
y_train_cat = keras.utils.to_categorical(y_train, 10)
y_test_cat = keras.utils.to_categorical(y_test, 10)
# Print the first few rows before and after one-hot encoding
print("Original labels:", y_train[:5])
print("One-hot encoded labels:\n", y_train_cat[:5])
```

```
[44]: # Code for your answer here!
from tensorflow import keras
from tensorflow.keras import layers

y_train_oh = keras.utils.to_categorical(y_train, 10)
```



```

y_test_oh = keras.utils.to_categorical(y_test, 10)
model_nn = keras.Sequential([
    layers.Dense(128, activation='relu', input_shape=(X_train.shape[1],)),
    layers.Dense(10, activation='softmax')
])

model_nn.compile(
    optimizer='adam',
    loss='categorical_crossentropy',
    metrics=['accuracy']
)

model_nn.summary()
history = model_nn.fit(
    X_train,
    y_train_oh,
    epochs=10,
    batch_size=128,
    validation_split=0.1
)

plt.plot(history.history['loss'], label='train loss')
plt.plot(history.history['val_loss'], label='val loss')
plt.legend()
plt.title("Loss")
plt.show()

plt.plot(history.history['accuracy'], label='train acc')
plt.plot(history.history['val_accuracy'], label='val acc')
plt.legend()
plt.title("Accuracy")
plt.show()

y_pred_prob = model_nn.predict(X_test)
y_pred = np.argmax(y_pred_prob, axis=1)

cm = confusion_matrix(y_test, y_pred)

sns.heatmap(cm, annot=True, fmt='d', cmap='Blues')
plt.title("Confusion Matrix")
plt.xlabel("Predicted")
plt.ylabel("True")
plt.show()

print(classification_report(y_test, y_pred))
test_loss, test_acc = model_nn.evaluate(X_test, y_test_oh)
print("Test cross-entropy loss:", test_loss)
true_probs = y_pred_prob[np.arange(len(y_test)), y_test]

```

```

correct_probs = true_probs[y_pred == y_test]
wrong_probs = true_probs[y_pred != y_test]
plt.hist(correct_probs, bins=30, alpha=0.5, label='Correct')
plt.hist(wrong_probs, bins=30, alpha=0.5, label='Wrong')
plt.legend()
plt.show()

```

```

/opt/conda/lib/python3.12/site-packages/keras/src/layers/core/dense.py:106:
UserWarning: Do not pass an `input_shape`/`input_dim` argument to a layer. When
using Sequential models, prefer using an `Input(shape)` object as the first
layer in the model instead.

```

```

super().__init__(activity_regularizer=activity_regularizer, **kwargs)

```

Model: "sequential_2"

Layer (type)	Output Shape	Param #
dense_15 (Dense)	(None, 128)	100,480
dense_16 (Dense)	(None, 10)	1,290

Total params: 101,770 (397.54 KB)

Trainable params: 101,770 (397.54 KB)

Non-trainable params: 0 (0.00 B)

Epoch 1/10

422/422 4s 7ms/step -

accuracy: 0.8971 - loss: 0.3722 - val_accuracy: 0.9585 - val_loss: 0.1627

Epoch 2/10

422/422 3s 8ms/step -

accuracy: 0.9514 - loss: 0.1696 - val_accuracy: 0.9660 - val_loss: 0.1216

Epoch 3/10

422/422 2s 6ms/step -

accuracy: 0.9646 - loss: 0.1228 - val_accuracy: 0.9705 - val_loss: 0.1028

Epoch 4/10

422/422 3s 6ms/step -

accuracy: 0.9725 - loss: 0.0954 - val_accuracy: 0.9733 - val_loss: 0.0924

Epoch 5/10

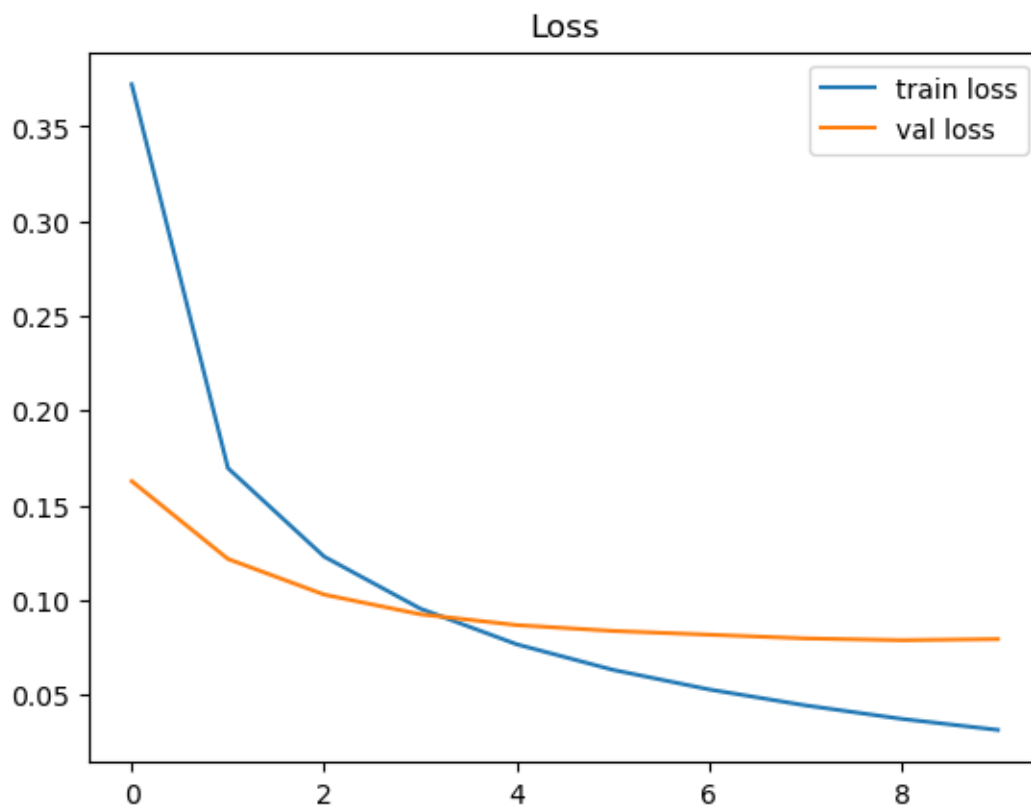
422/422 3s 6ms/step -

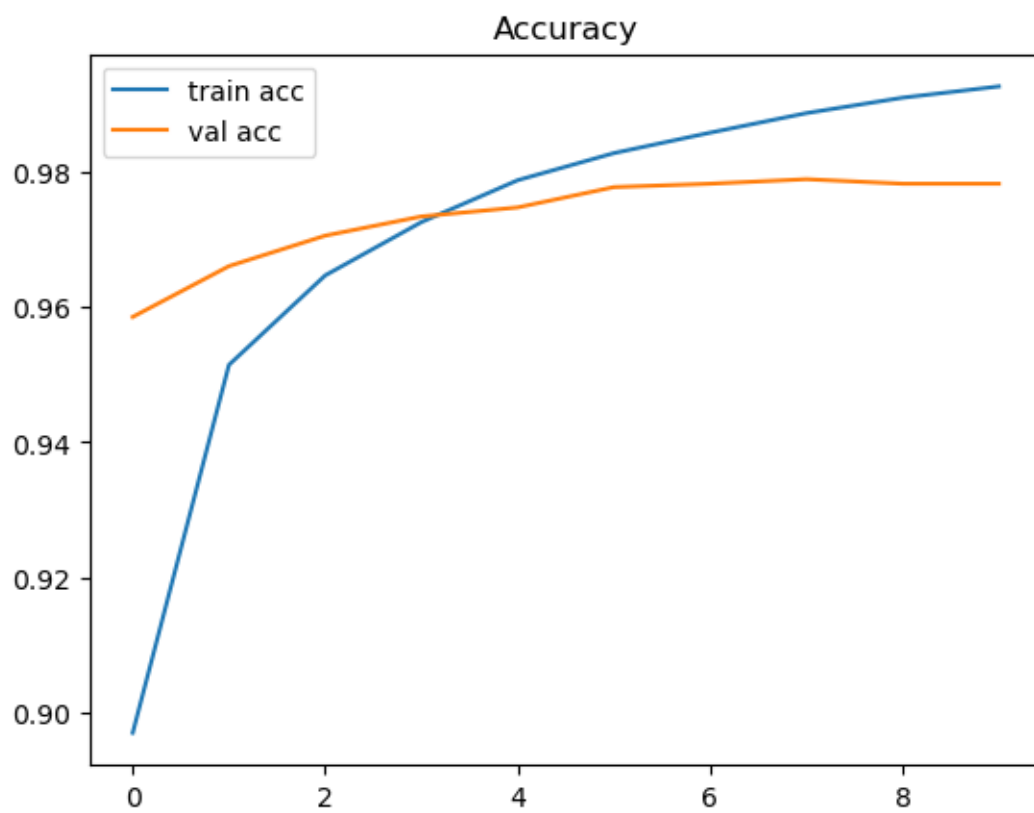
accuracy: 0.9787 - loss: 0.0765 - val_accuracy: 0.9747 - val_loss: 0.0867

Epoch 6/10

422/422 3s 6ms/step -

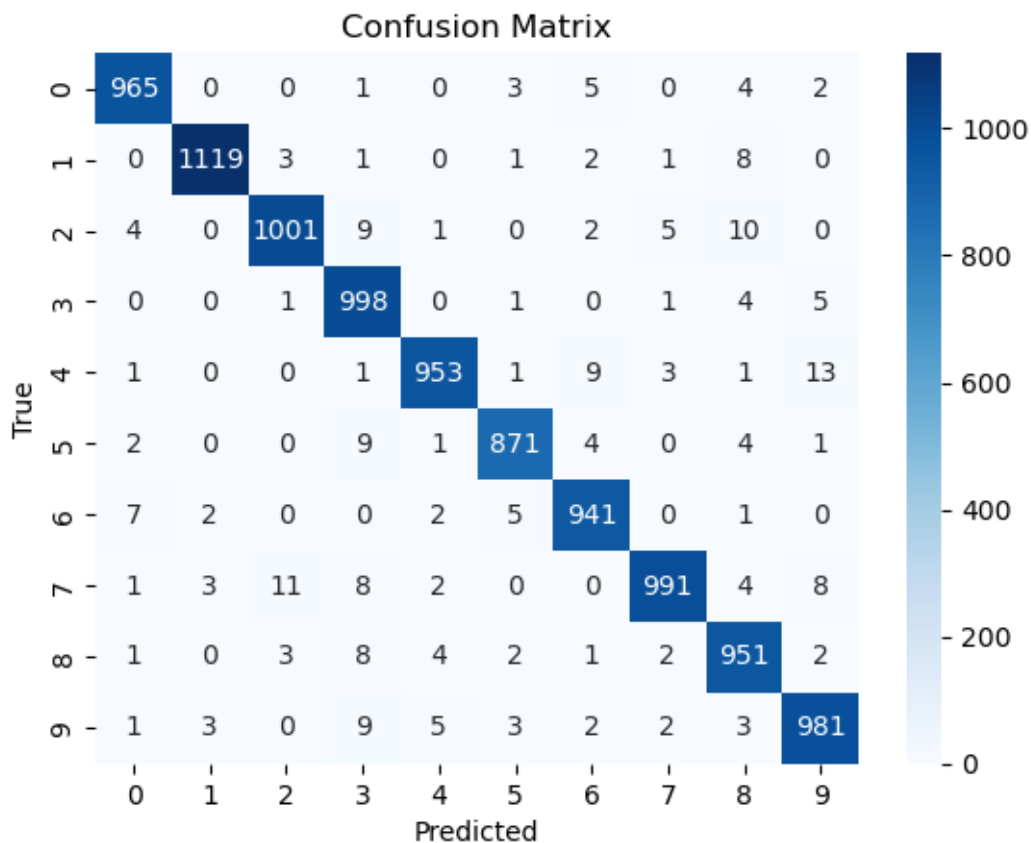
accuracy: 0.9827 - loss: 0.0631 - val_accuracy: 0.9777 - val_loss: 0.0836
Epoch 7/10
422/422 3s 6ms/step -
accuracy: 0.9857 - loss: 0.0527 - val_accuracy: 0.9782 - val_loss: 0.0816
Epoch 8/10
422/422 3s 6ms/step -
accuracy: 0.9886 - loss: 0.0443 - val_accuracy: 0.9788 - val_loss: 0.0797
Epoch 9/10
422/422 2s 5ms/step -
accuracy: 0.9909 - loss: 0.0372 - val_accuracy: 0.9782 - val_loss: 0.0787
Epoch 10/10
422/422 2s 6ms/step -
accuracy: 0.9925 - loss: 0.0313 - val_accuracy: 0.9782 - val_loss: 0.0793





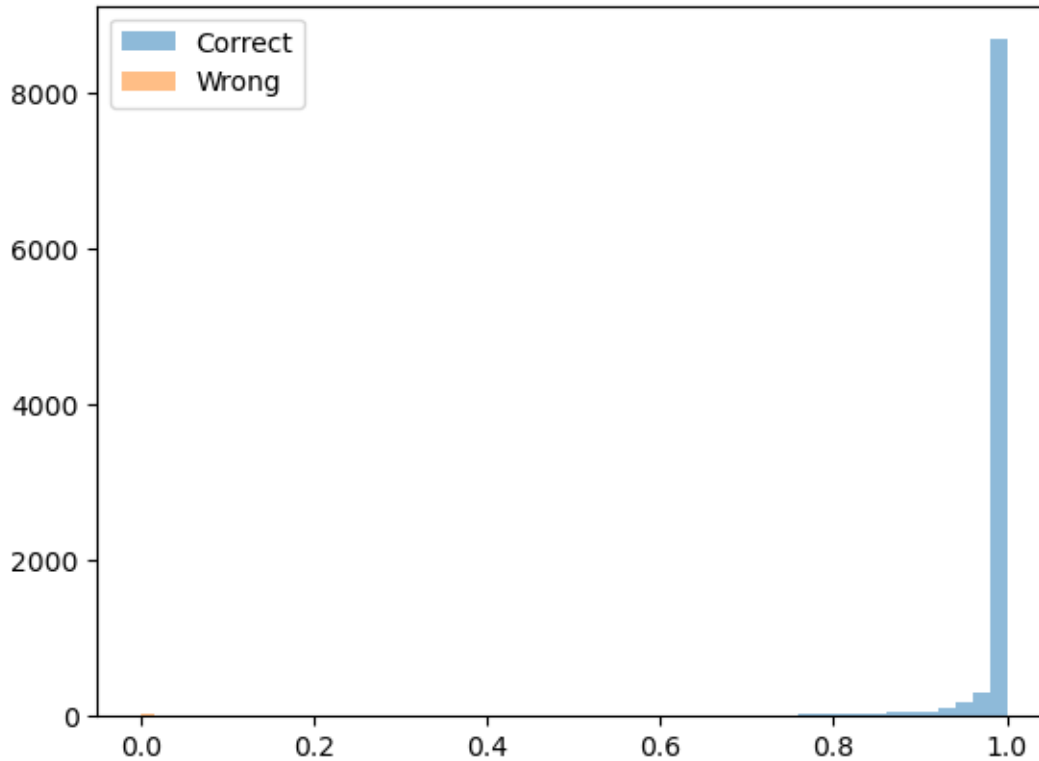
313/313

1s 2ms/step



	precision	recall	f1-score	support
0	0.98	0.98	0.98	980
1	0.99	0.99	0.99	1135
2	0.98	0.97	0.98	1032
3	0.96	0.99	0.97	1010
4	0.98	0.97	0.98	982
5	0.98	0.98	0.98	892
6	0.97	0.98	0.98	958
7	0.99	0.96	0.97	1028
8	0.96	0.98	0.97	974
9	0.97	0.97	0.97	1009
accuracy			0.98	10000
macro avg	0.98	0.98	0.98	10000
weighted avg	0.98	0.98	0.98	10000

313/313 1s 2ms/step -
accuracy: 0.9771 - loss: 0.0774
Test cross-entropy loss: 0.07744649052619934



4.0.4 Exercise 10 (CORE)

Comment on the following:

- Do you think you need to train the model for more epochs?
- Which model do you prefer: the baseline logistic regression or the neural network? Consider the trade-off between: the number of parameters and predictive performance.

no, because the loss not decreasing significantly if use more epochs. the neural network can give better performance although the model is more complex.

4.0.5 Exercise 11 (EXTRA)

Let's explore a deeper architecture.

- Build a deeper network with three hidden layers, with $D_1 = 128$, $D_2 = 64$, $D_3 = 32$ neurons and ReLU activation, and an output layer with 10 neurons and softmax activation.
- Compile and train the model with the same setting as the shallow model.
- Plot the loss and accuracy for the training and validation sets as a function of epochs. Do you think more epochs are needed?
- Plot the confusion matrix, print the classification report, and cross entropy loss. Visualize some the images, along with their probabilities for both correctly and incorrectly classified

images.

- Which model do you prefer? Consider the trade-off between: the number of parameters and predictive performance.

```
[ ]: # Code for your answer here!
```

Type your answer here

5 Completing the Worksheet

At this point you have hopefully been able to complete all the CORE exercises and attempted the EXTRA ones. Now is a good time to check the reproducibility of this document by restarting the notebook's kernel and rerunning all cells in order.

Before generating the PDF, please **change 'Student 1' and 'Student 2' at the top of the notebook to include your name(s).**

Once that is done and you are happy with everything, you can then run the following cell to generate your PDF. Once generated, please submit this PDF on Learn page by 16:00 PM on the Friday of the week the workshop was given.

```
[ ]: !jupyter nbconvert --to pdf mlp_week09.ipynb
```